

ADVERSARIAL SEARCH

Artificial Intelligence

AI Course
IIITDM Kurnool

Dr. Alapan Kuila
Assistant Professor

Lecture slides adapted from materials by

Stuart Russell • Mausam • Henry Kautz • Linda Shapiro

Game Playing

Why do AI researchers study game playing?

1. It's a good reasoning problem, formal and nontrivial.
2. Direct comparison with humans and other computer programs is easy.

What Kinds of Games?

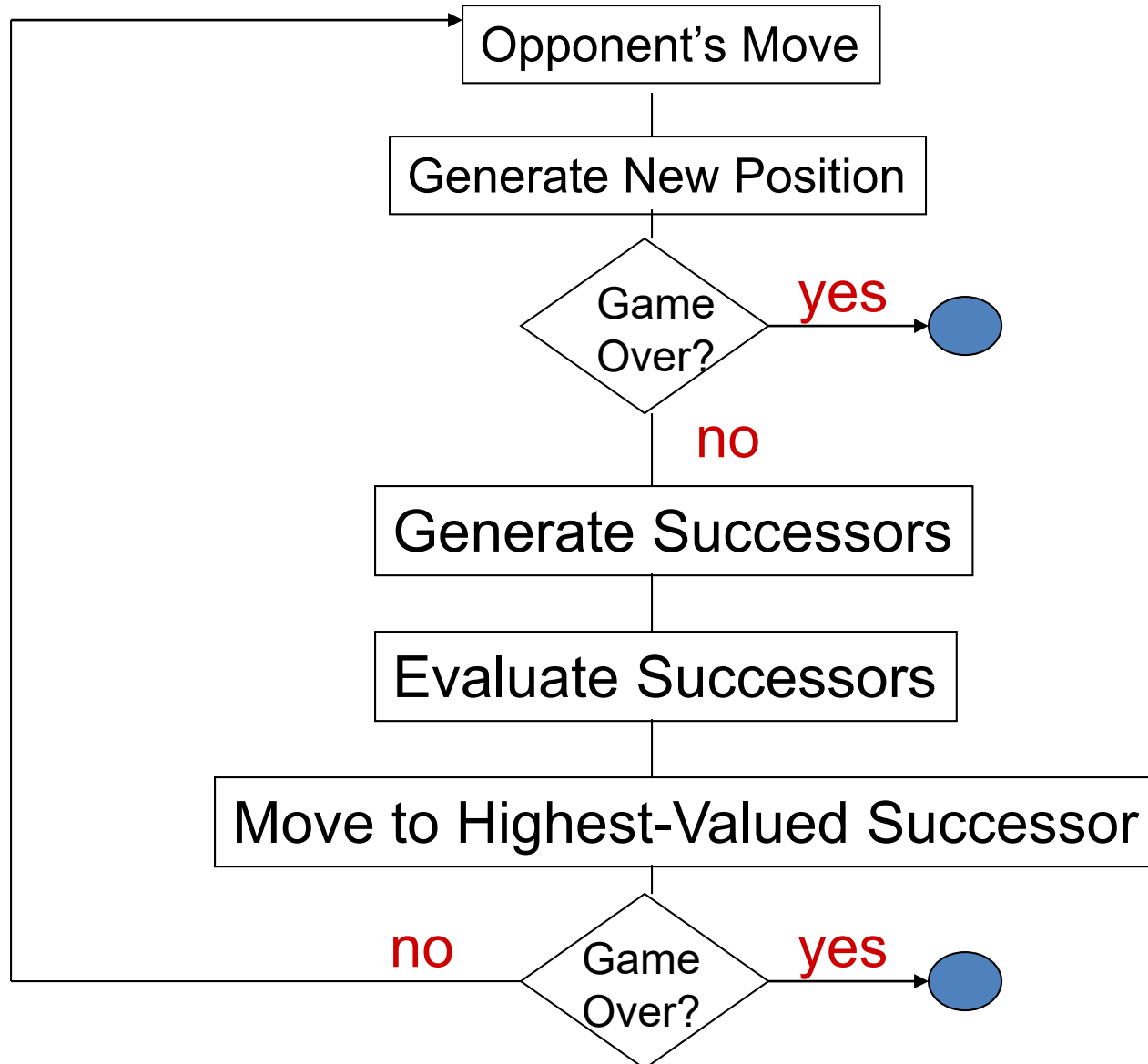
Mainly games of strategy with the following characteristics:

1. Sequence of **moves** to play
2. Rules that specify **possible moves**
3. Rules that specify a **payment** for each move
4. Objective is to **maximize** your payment

Games vs. Search Problems

- **Unpredictable opponent** → specifying a move for every possible opponent reply
- **Time limits** → unlikely to find goal, must approximate

Two-Player Game



Games as Adversarial Search

- States:
 - board configurations
- Initial state:
 - the board position and which player will move
- Successor function:
 - returns list of (move, state) pairs, each indicating a legal move and the resulting state
- Terminal test:
 - determines when the game is over
- Utility function:
 - gives a numeric value in terminal states
(e.g., -1, 0, +1 for loss, tie, win)

Game Tree (2-player, Deterministic, Turns)

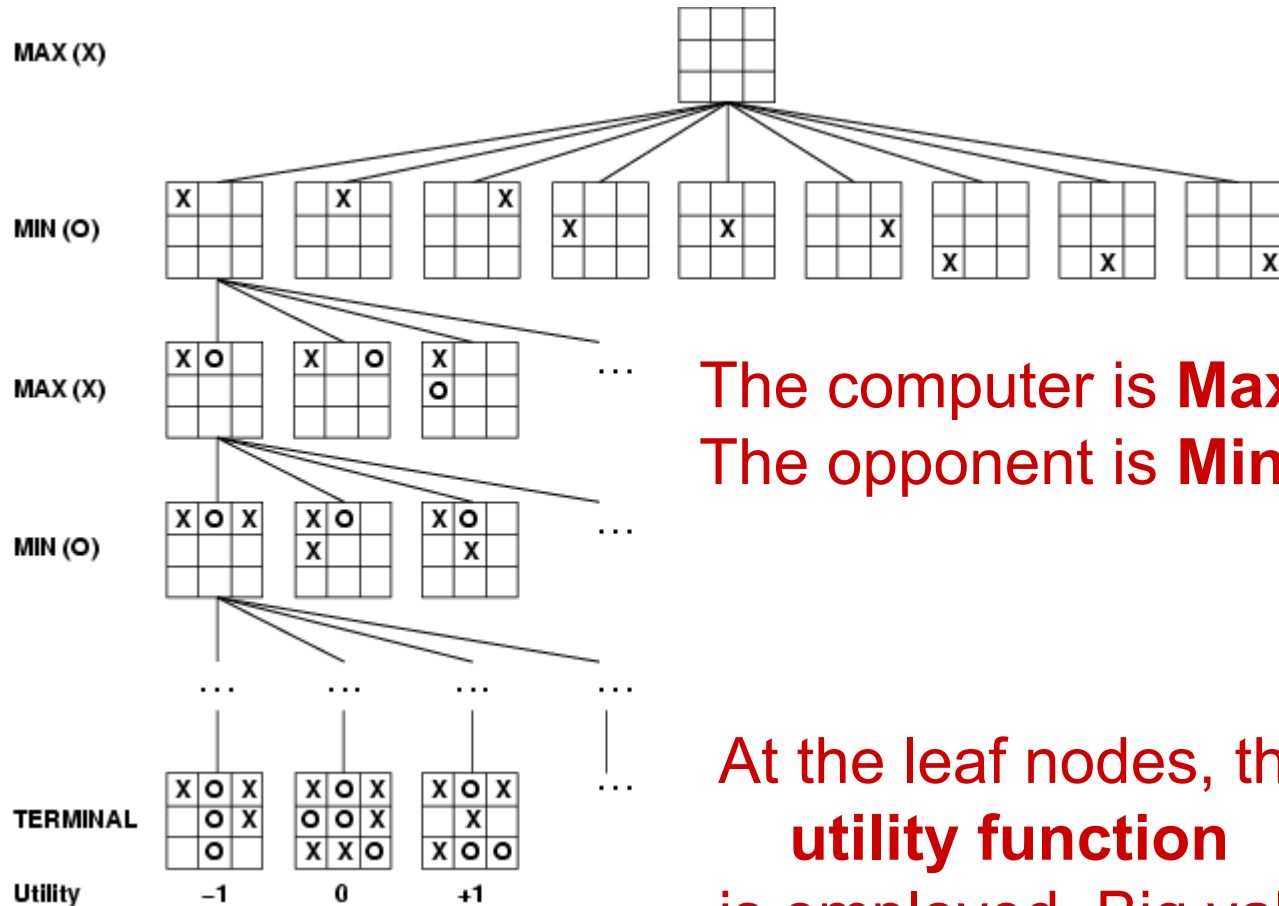
computer's
turn

opponent's
turn

computer's turn

opponent's turn

leaf nodes
are evaluated



The computer is **Max**.
The opponent is **Min**.

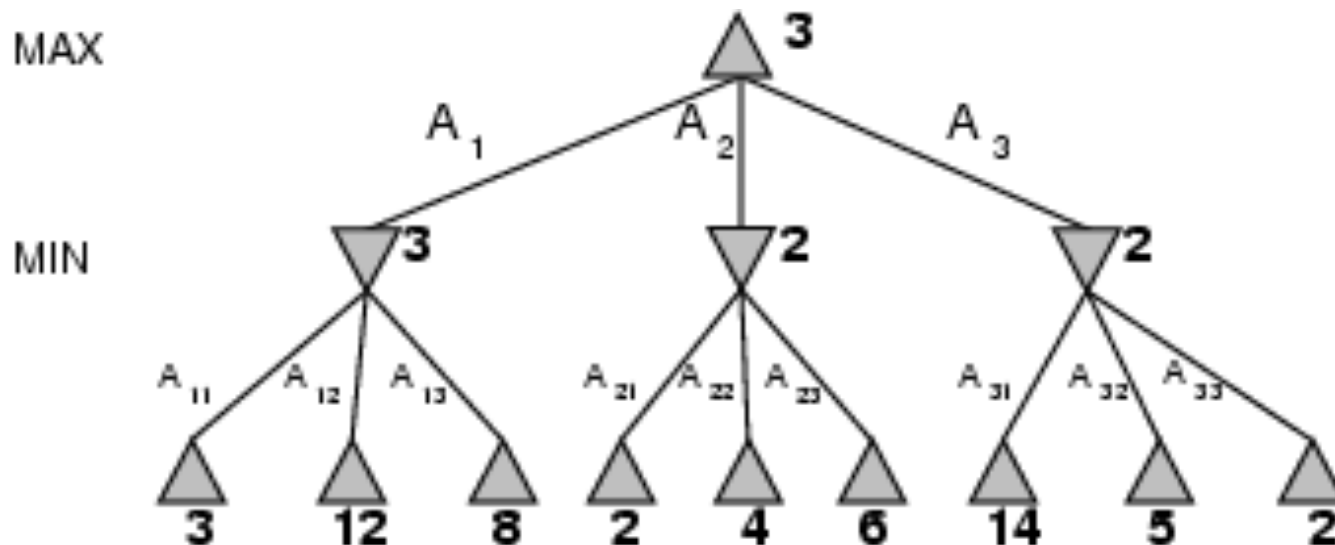
At the leaf nodes, the **utility function** is employed. Big value means good, small is bad.

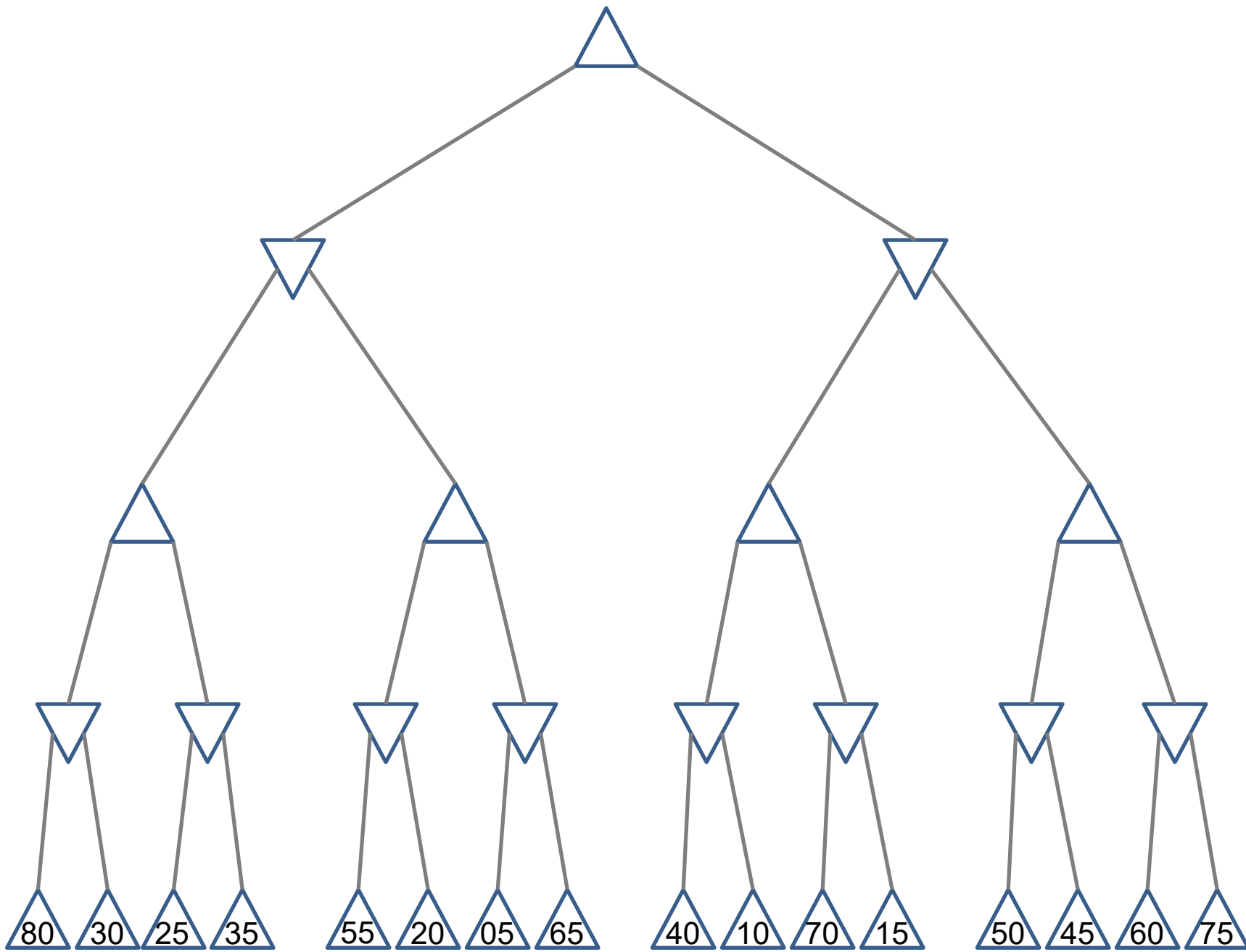
Mini-Max Terminology

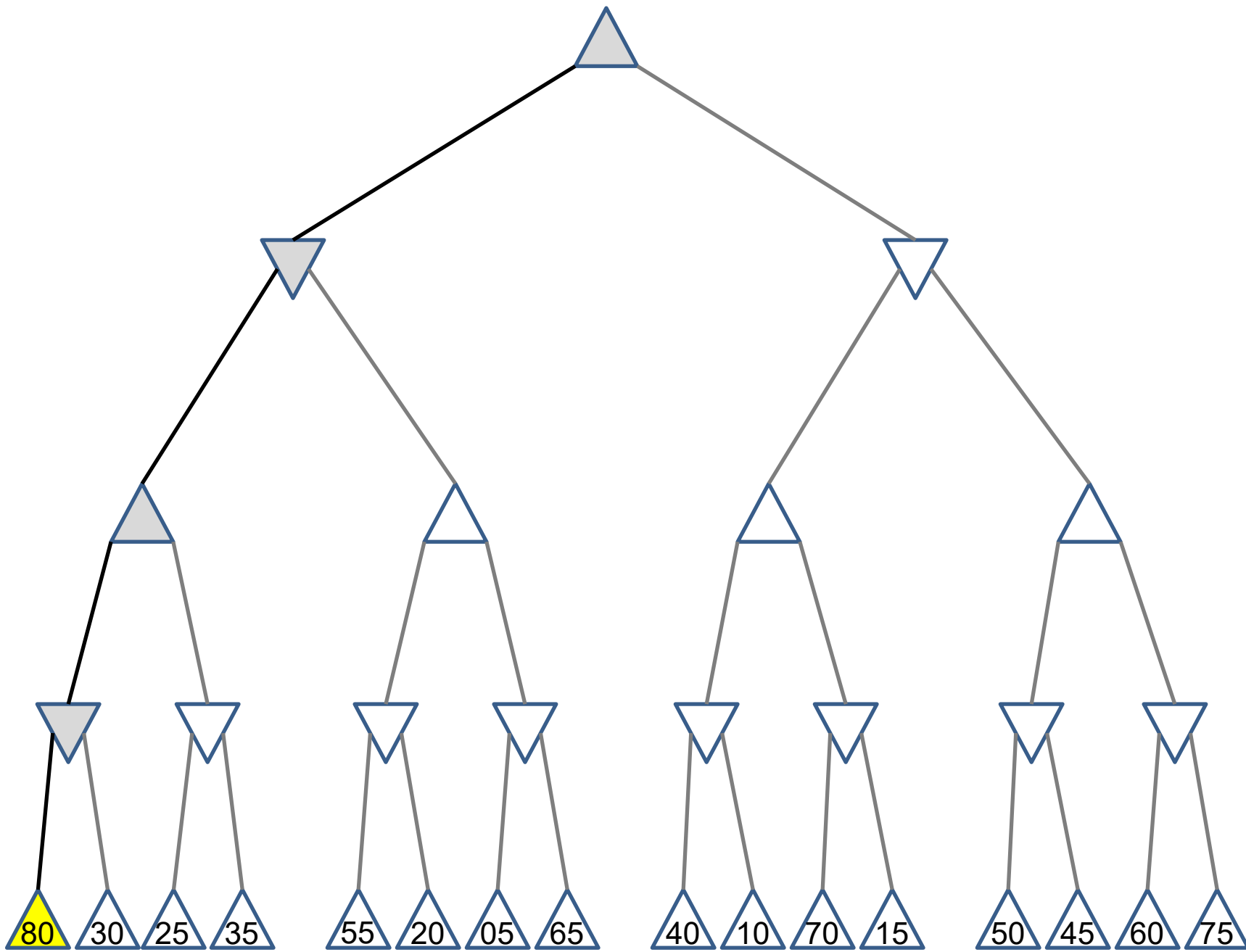
- **move**: a move by both players
- **ply**: a half-move
- **utility function**: the function applied to leaf nodes
- **backed-up value**
 - of a **max-position**: the value of its largest successor
 - of a **min-position**: the value of its smallest successor
- **minimax procedure**: search down several levels; at the bottom level apply the utility function, back-up values all the way up to the root node, and that node selects the move.

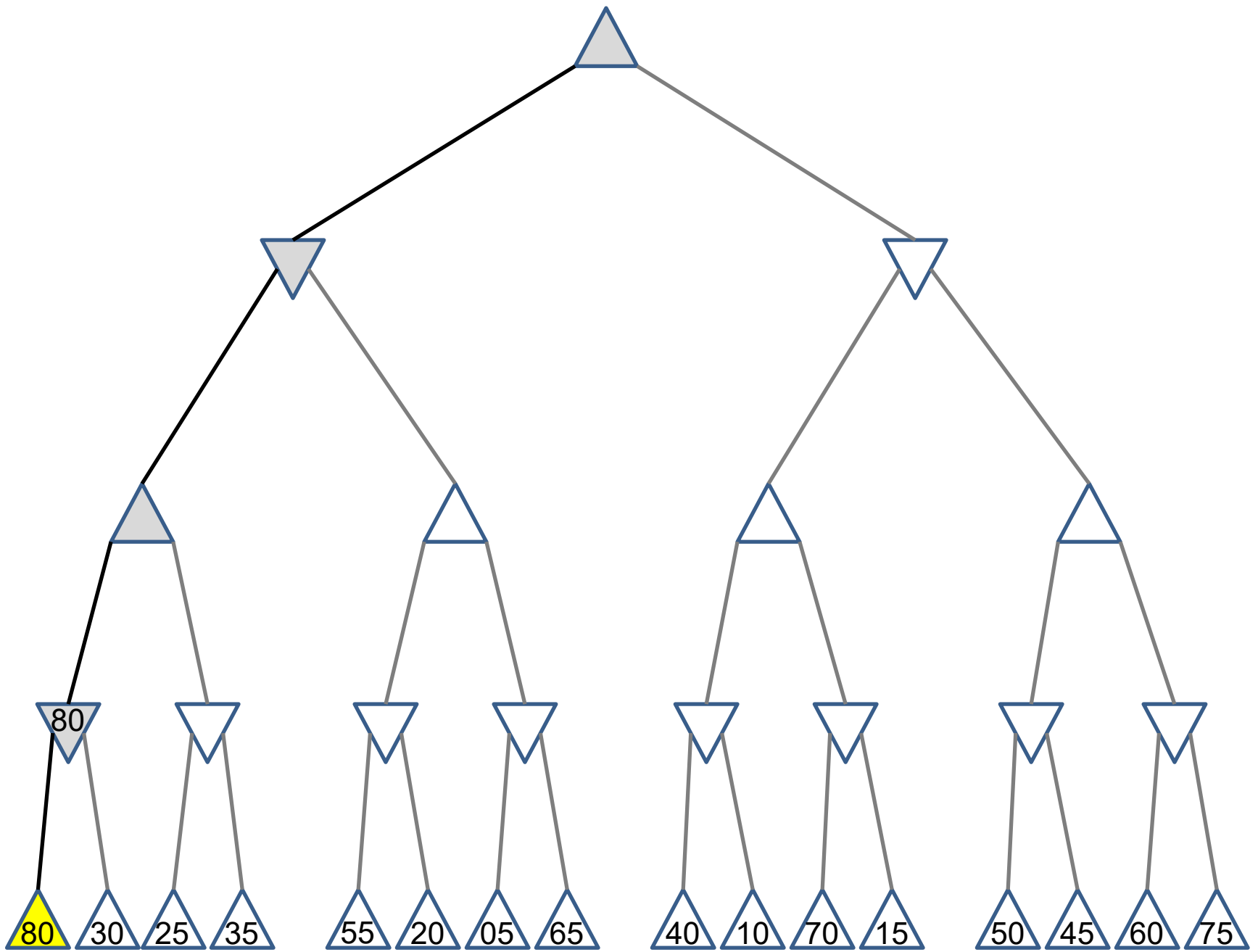
Minimax

- Perfect play for deterministic games
- Idea: choose move to position with highest **minimax value**
= best achievable payoff against best play
- E.g., 2-ply game:

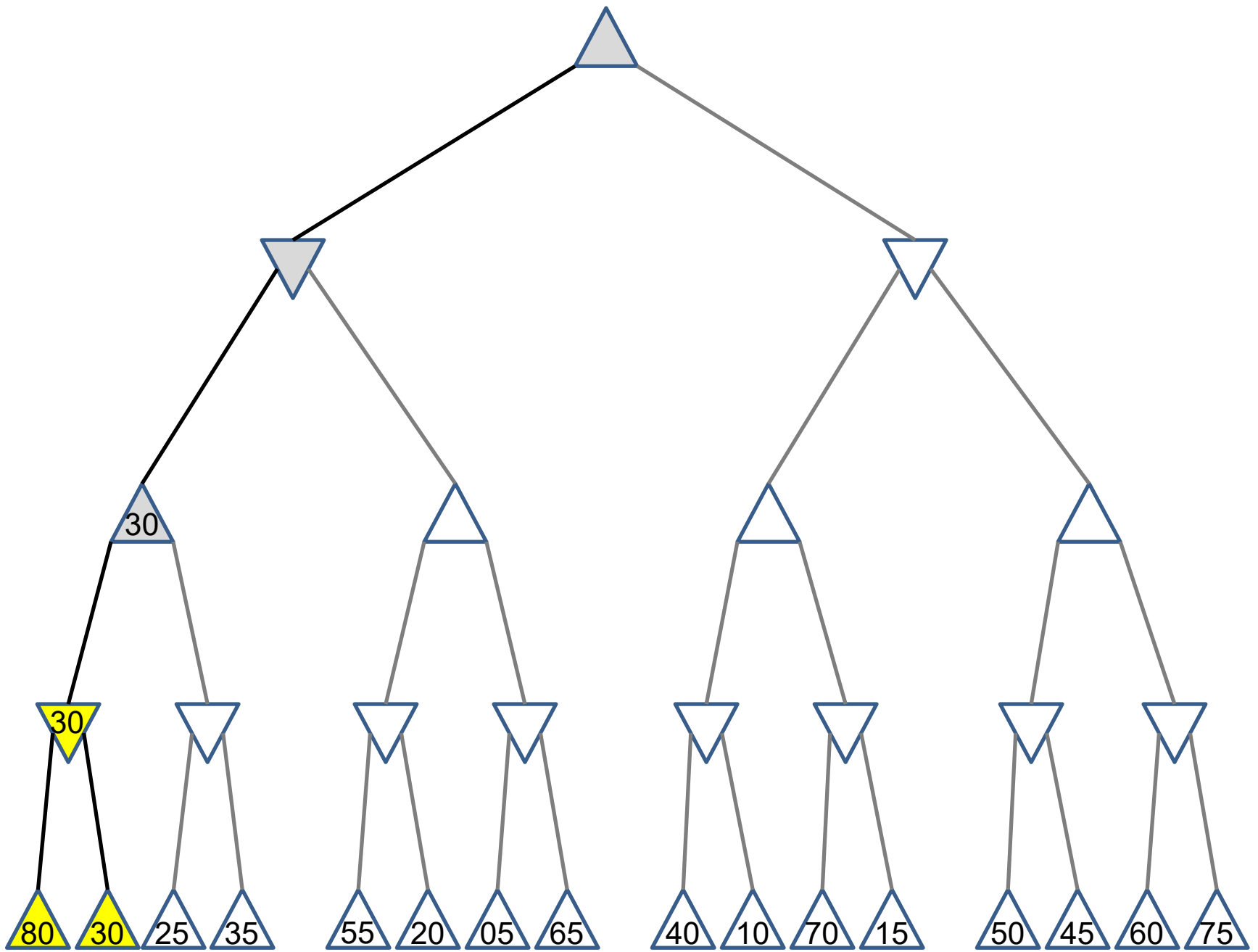


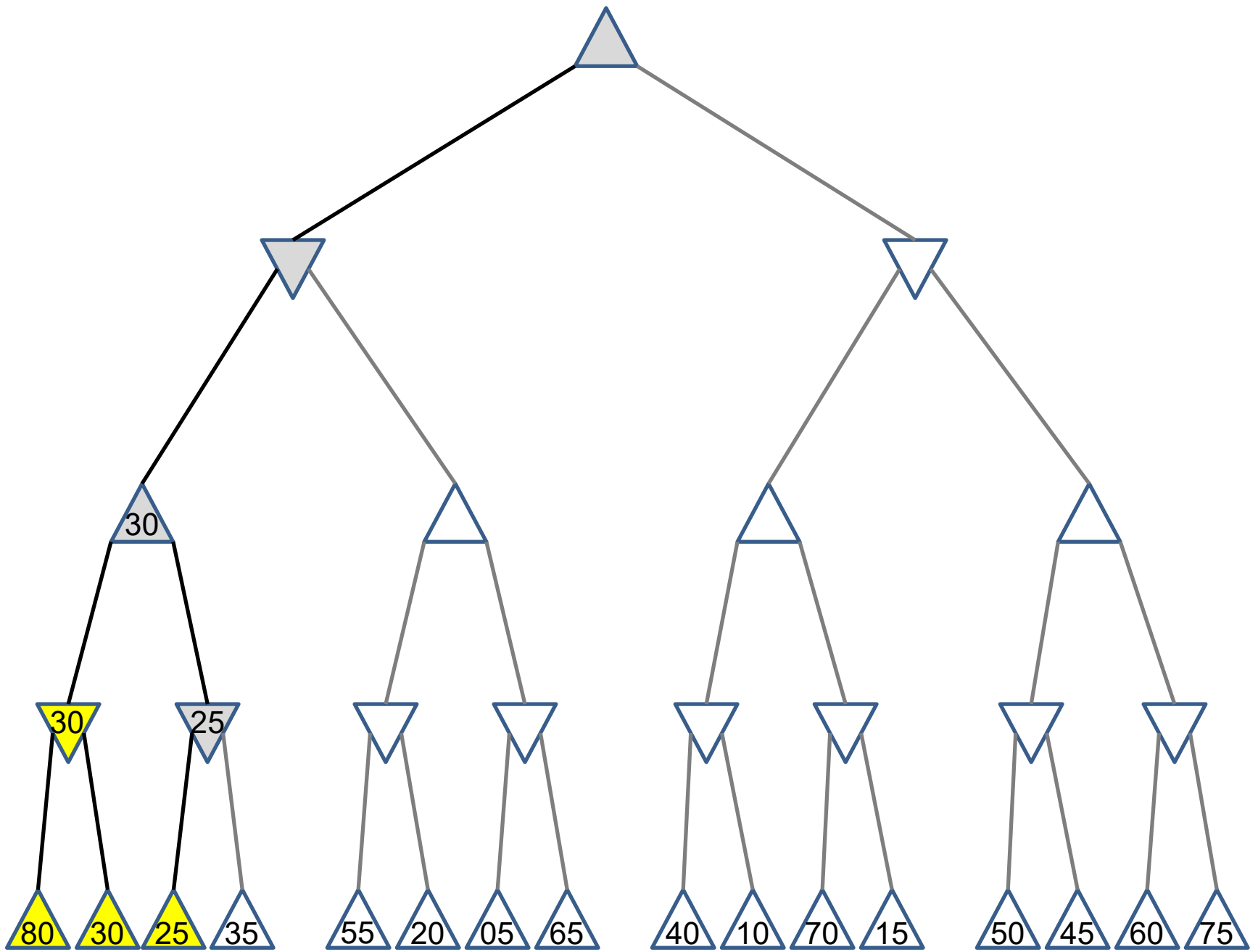


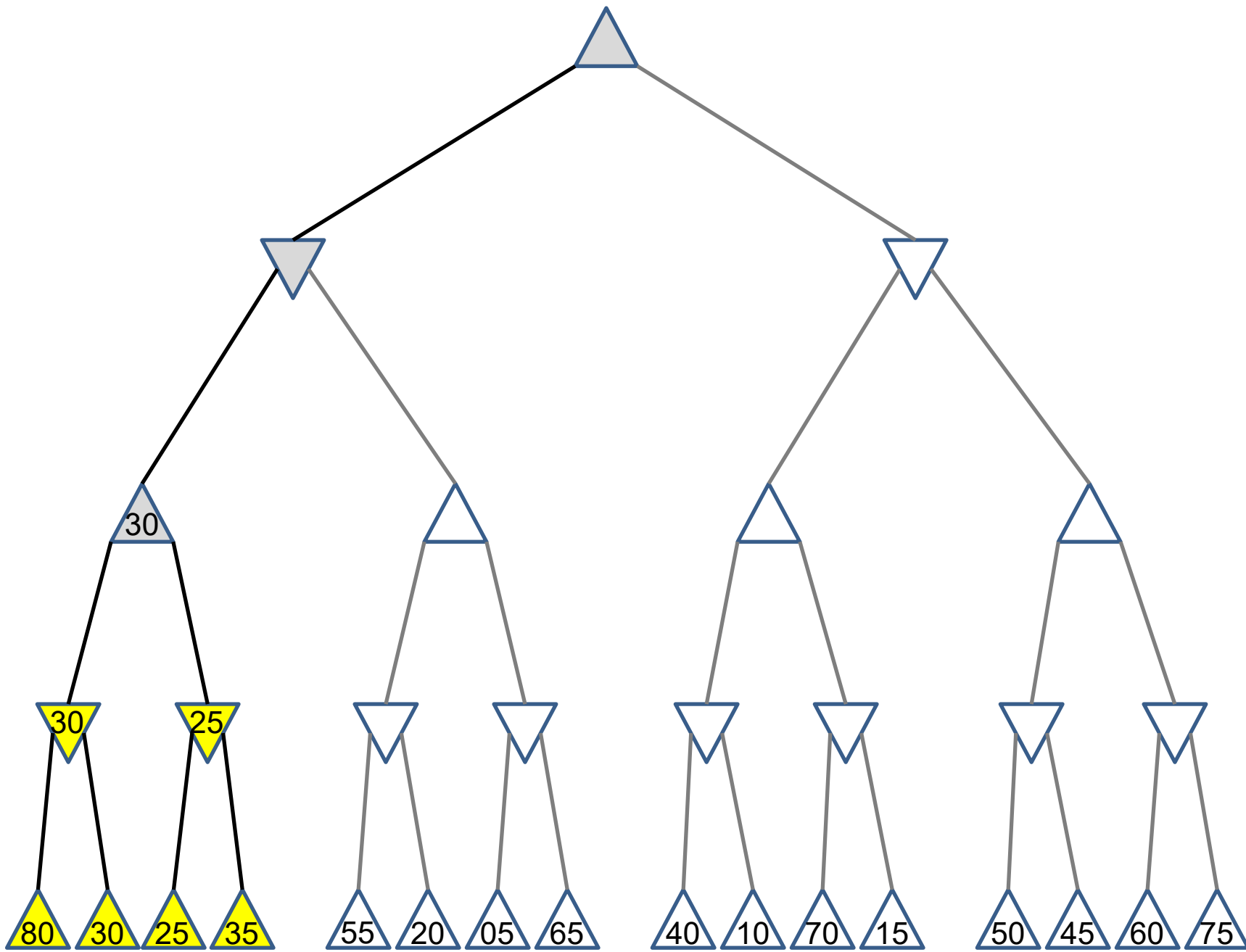


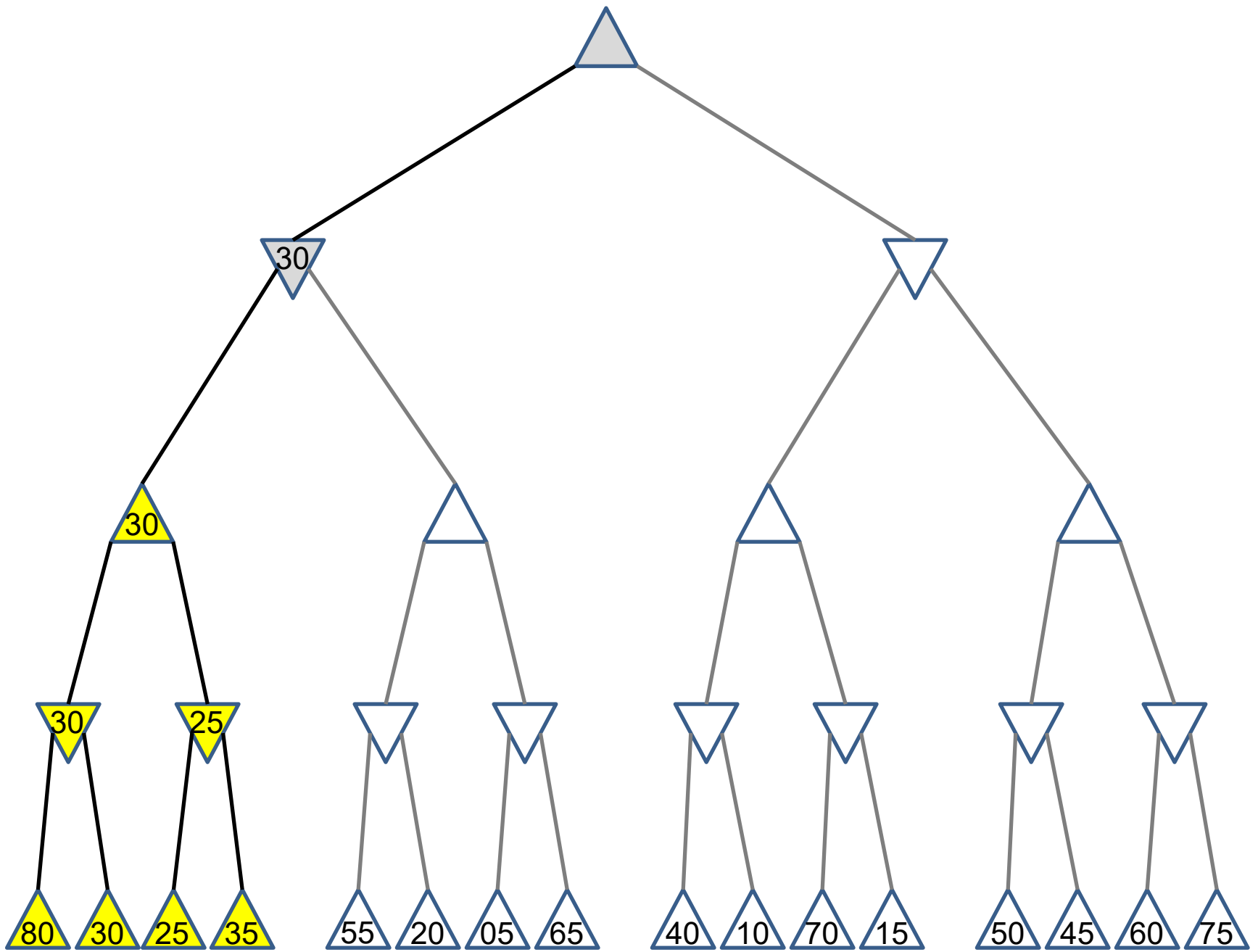


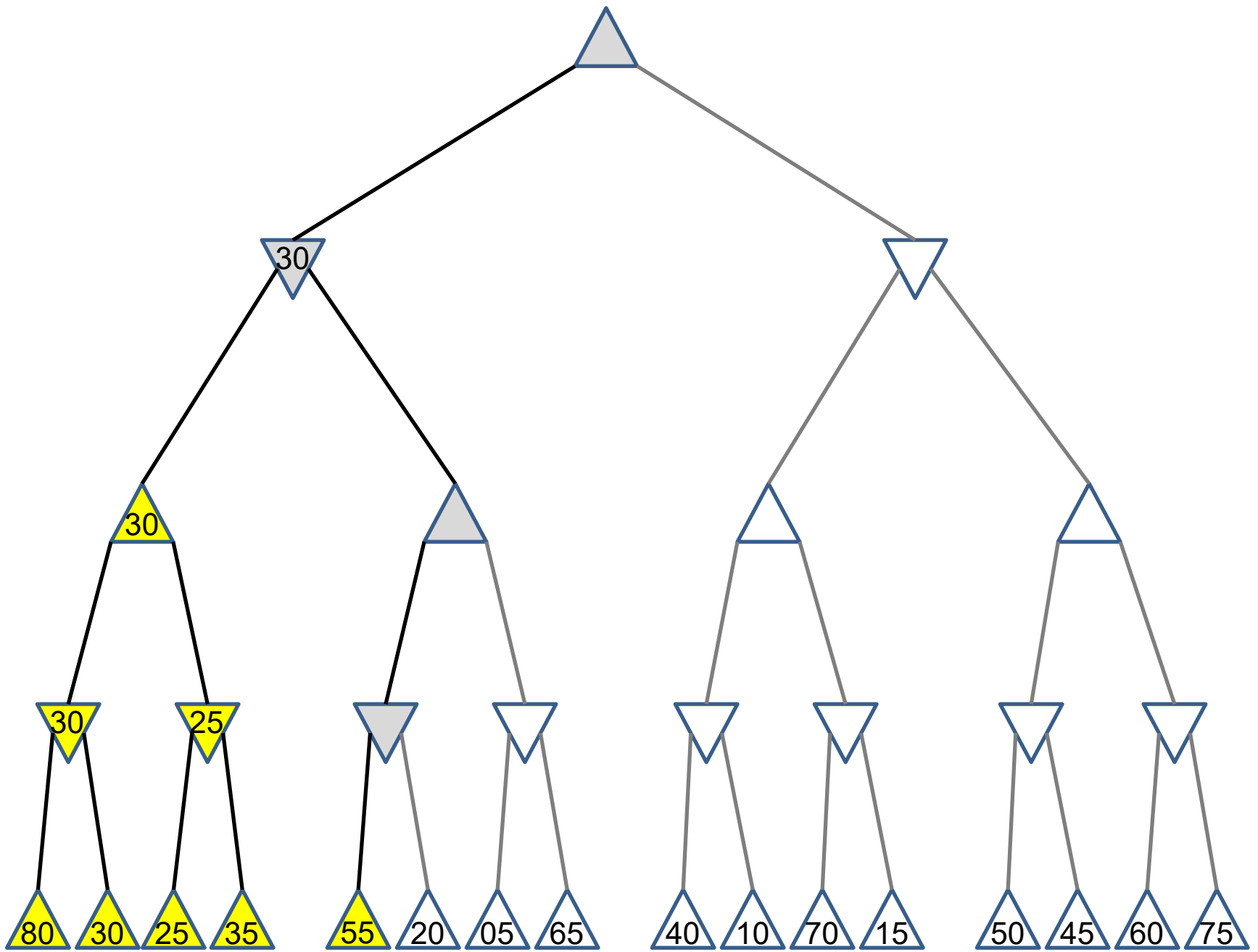


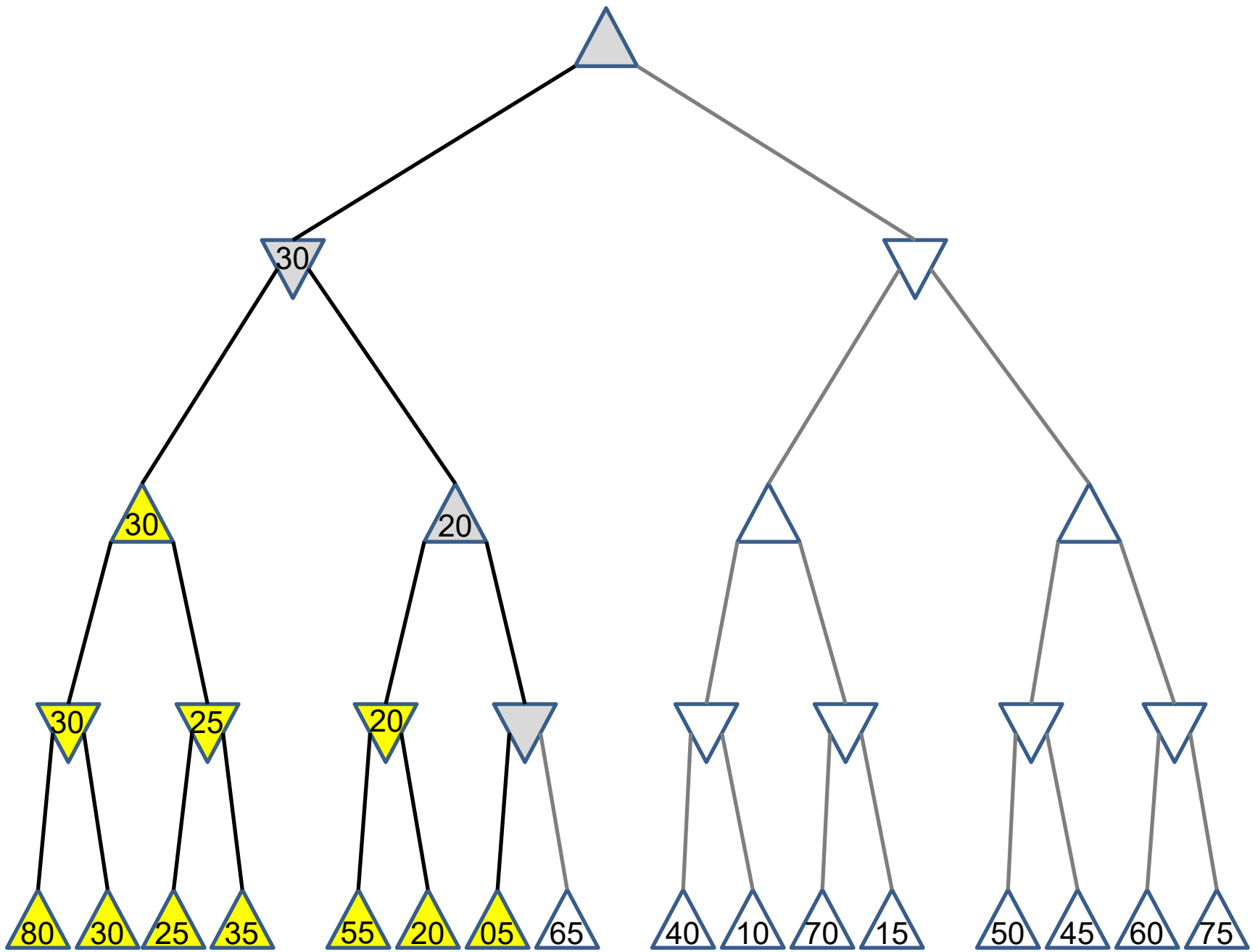


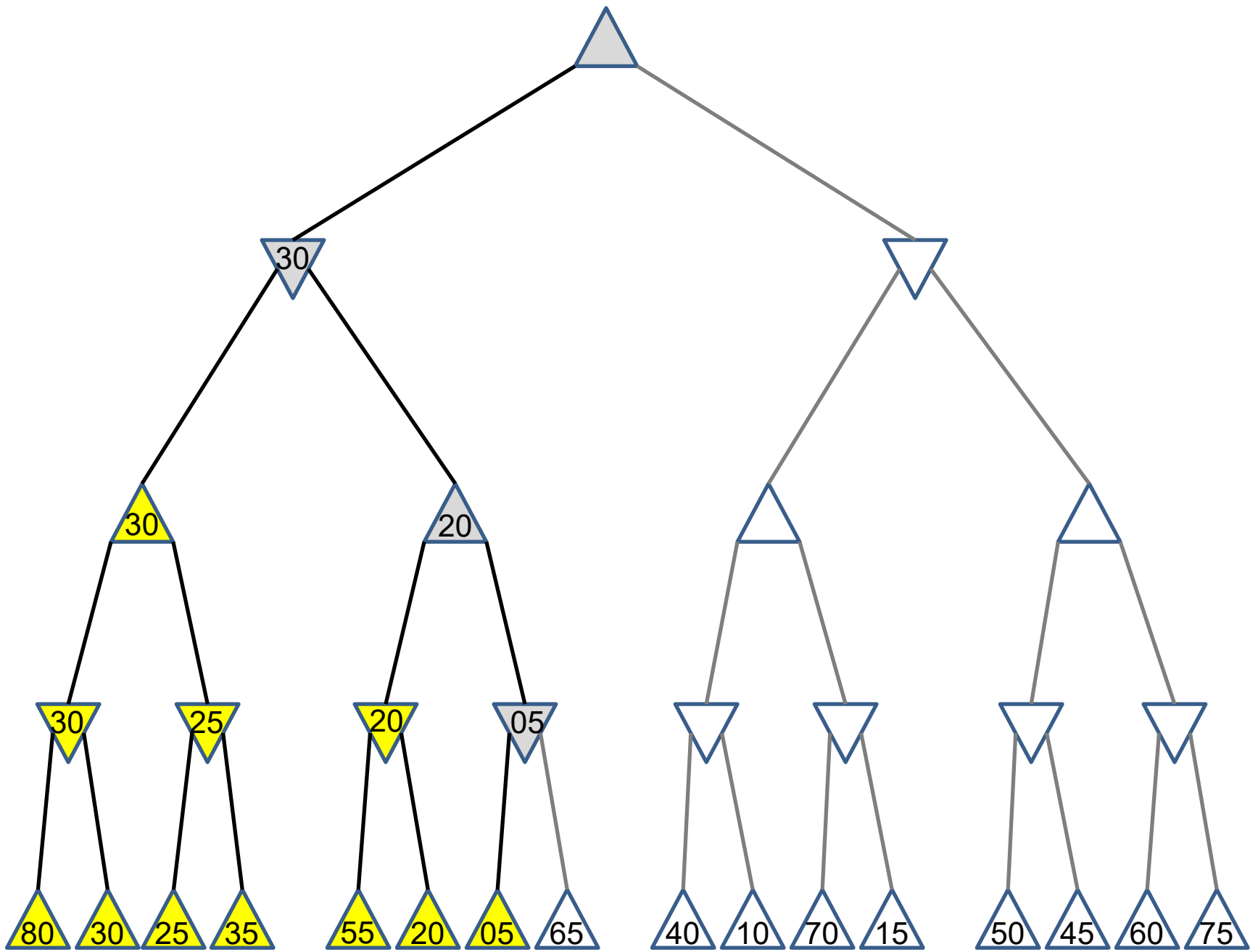


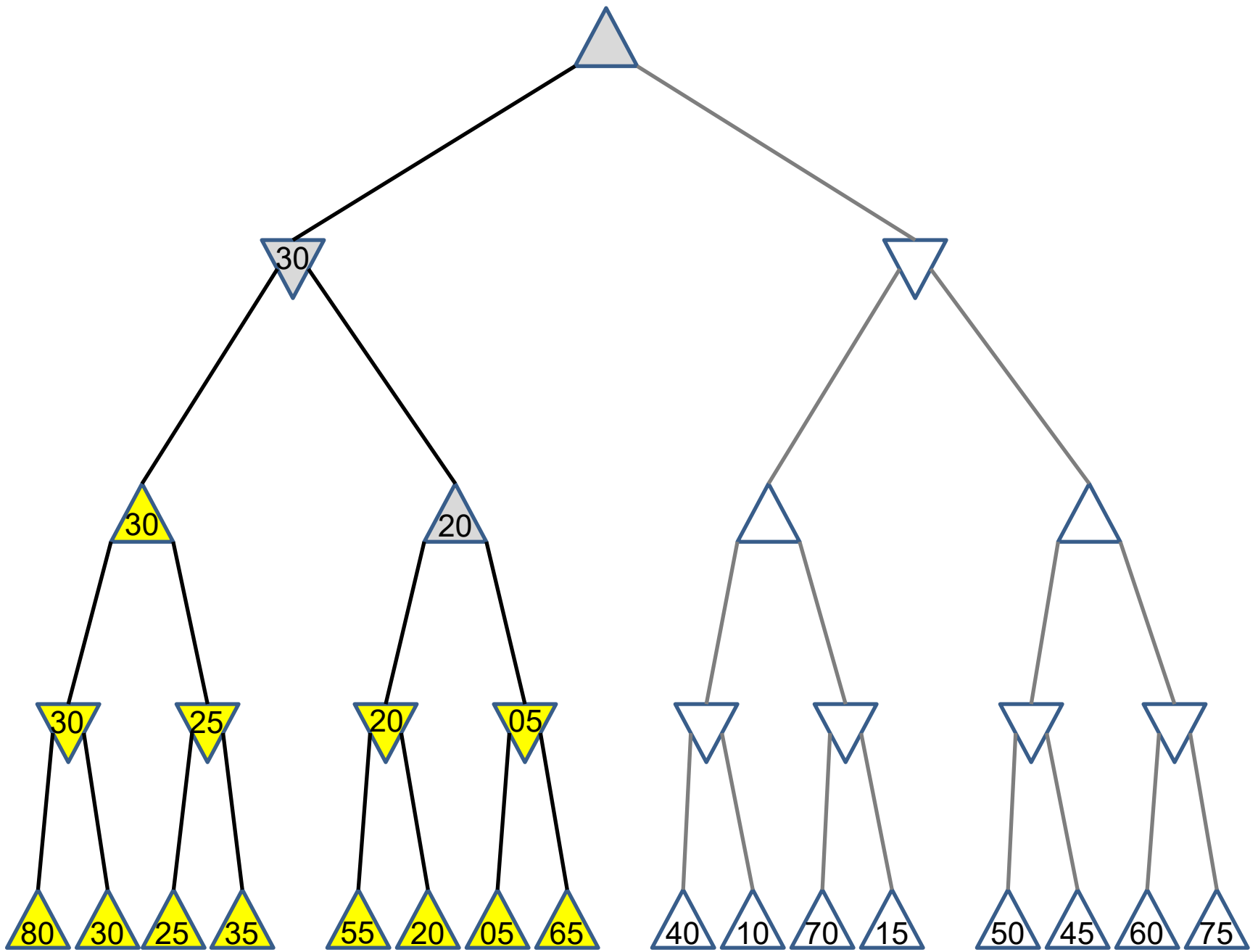


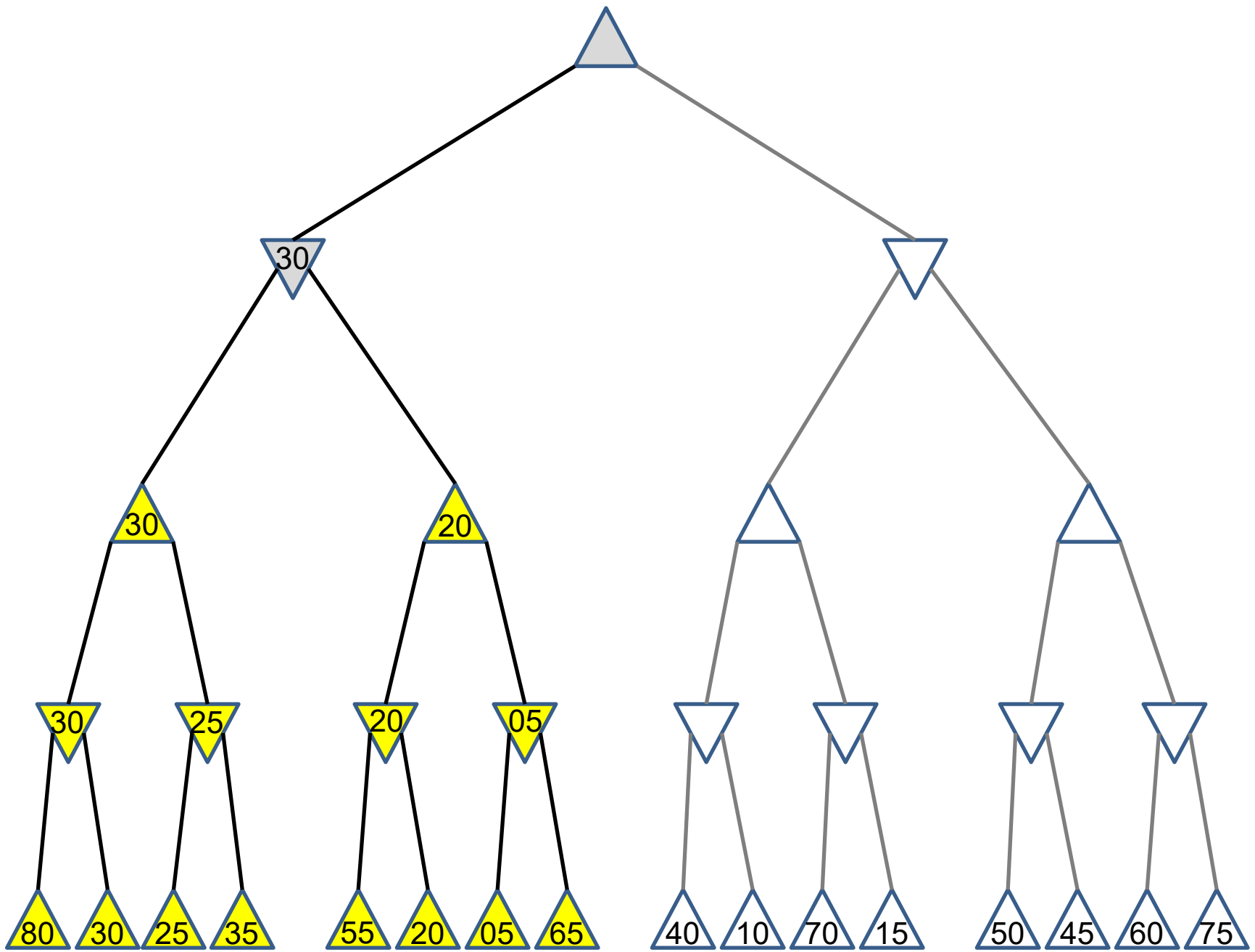


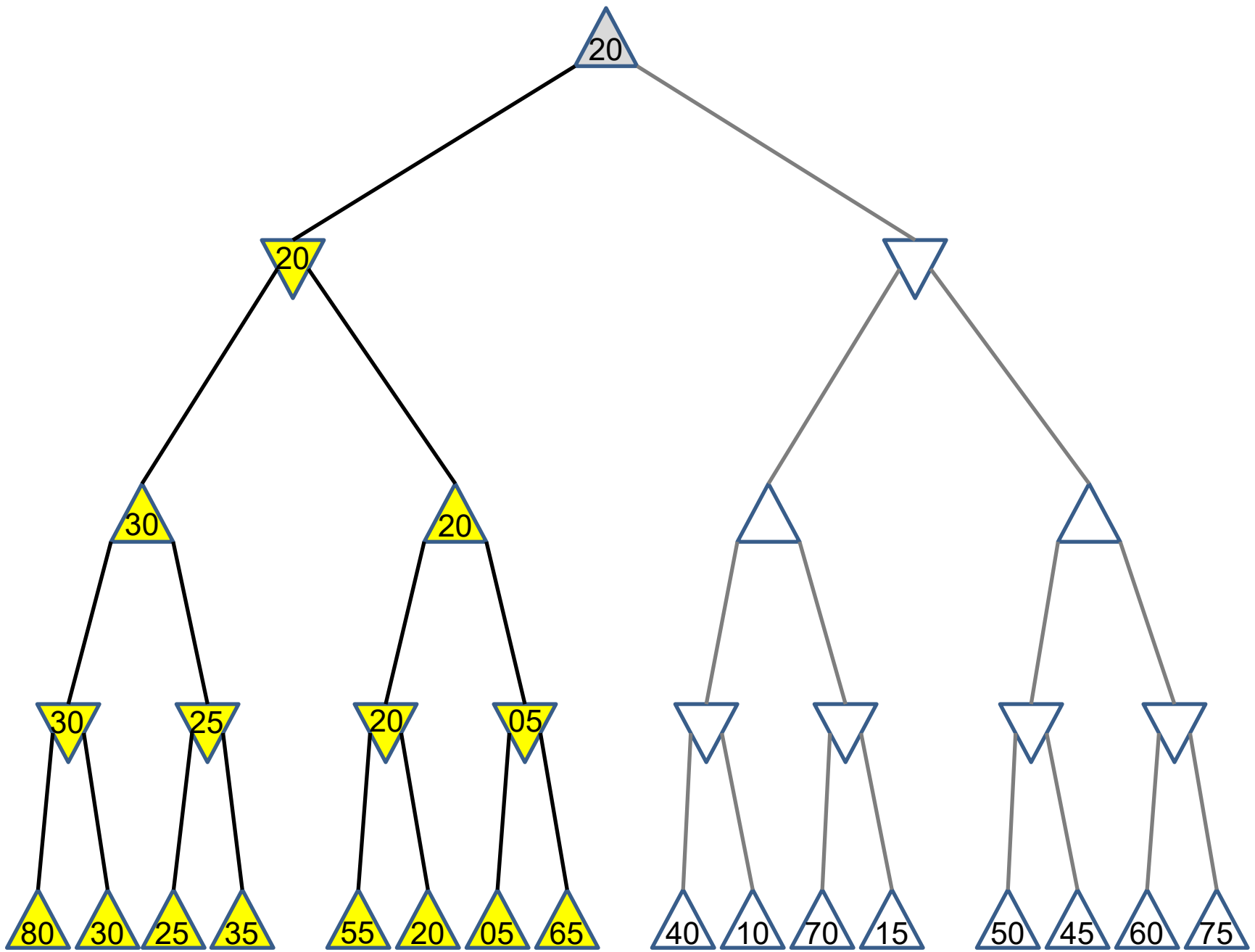


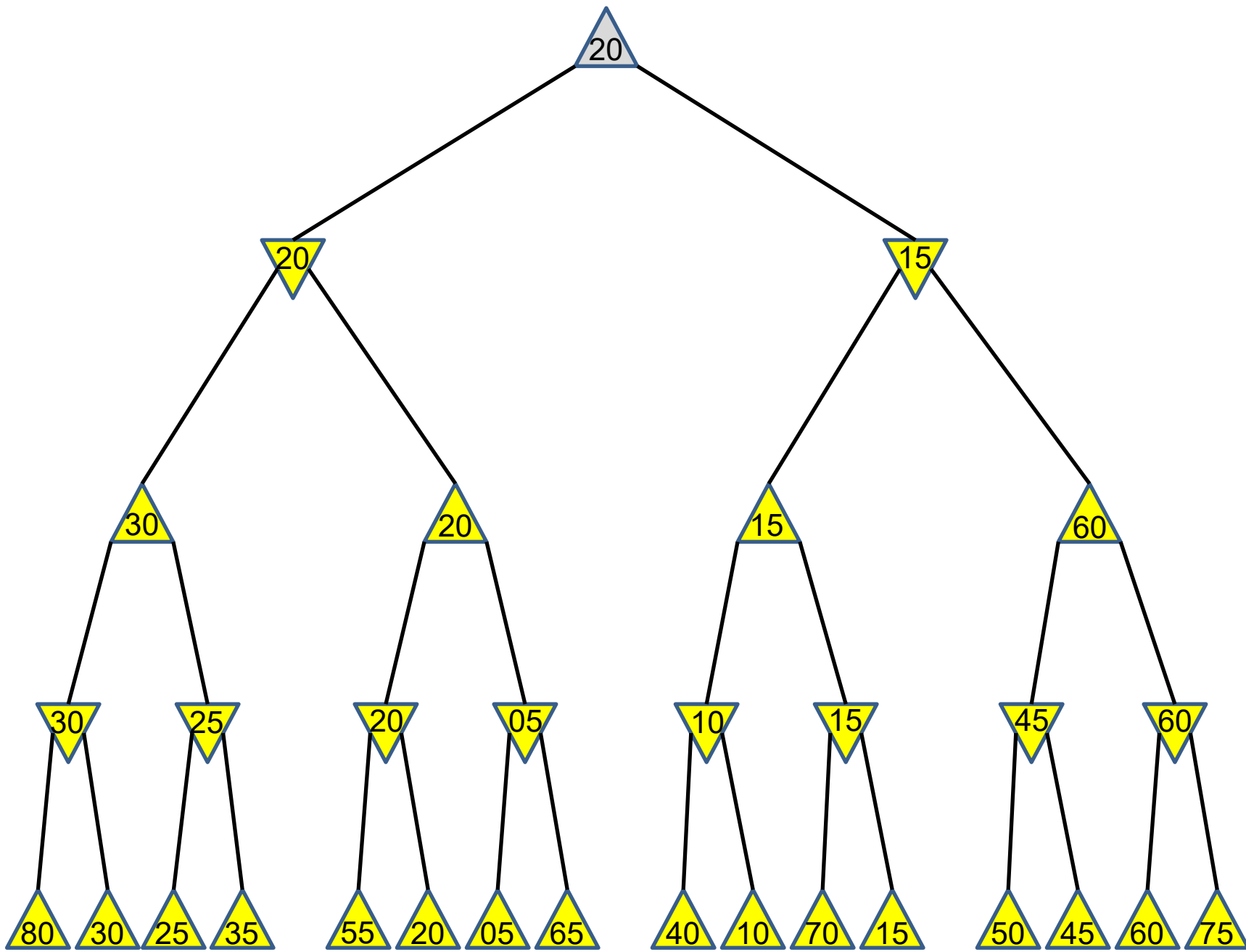


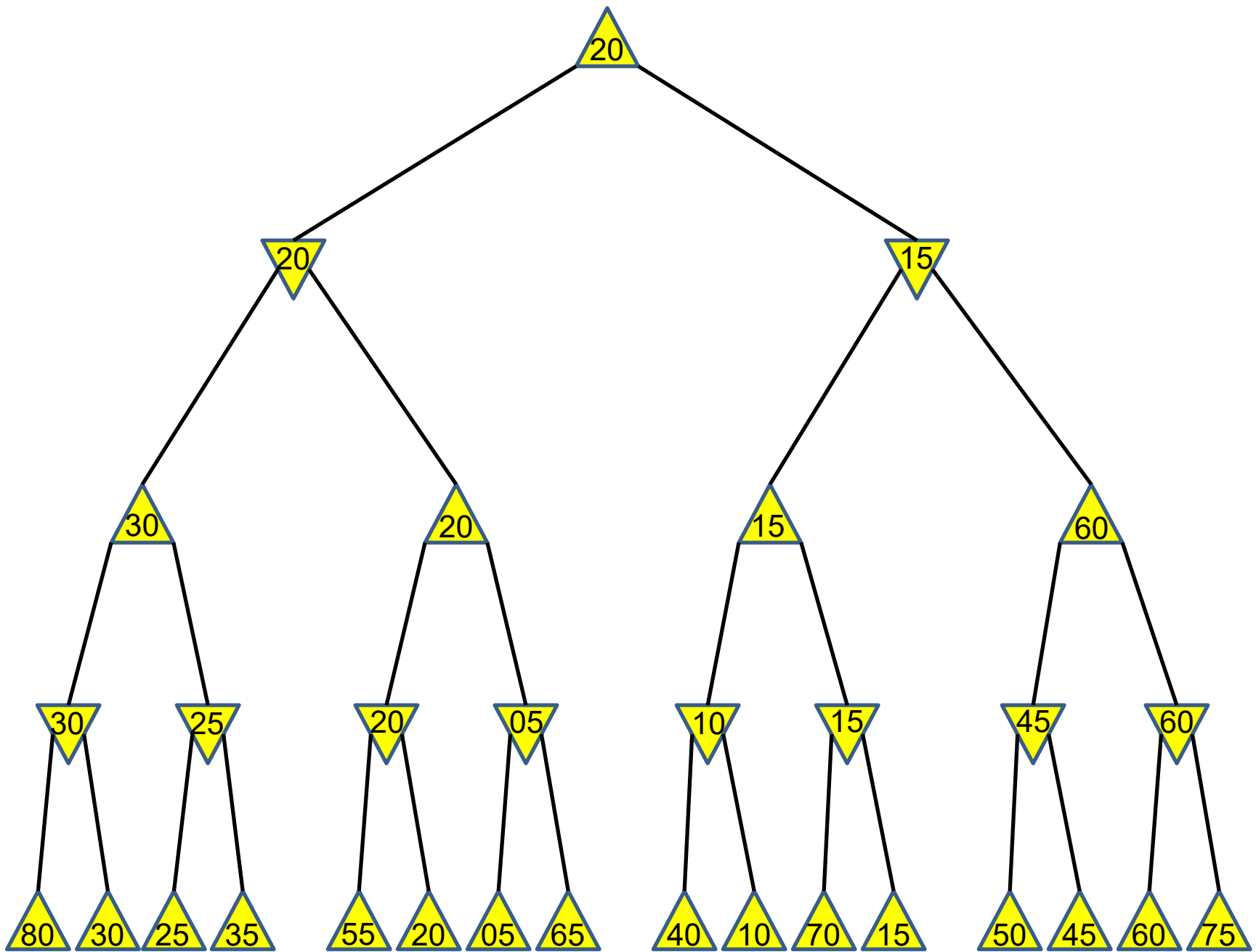


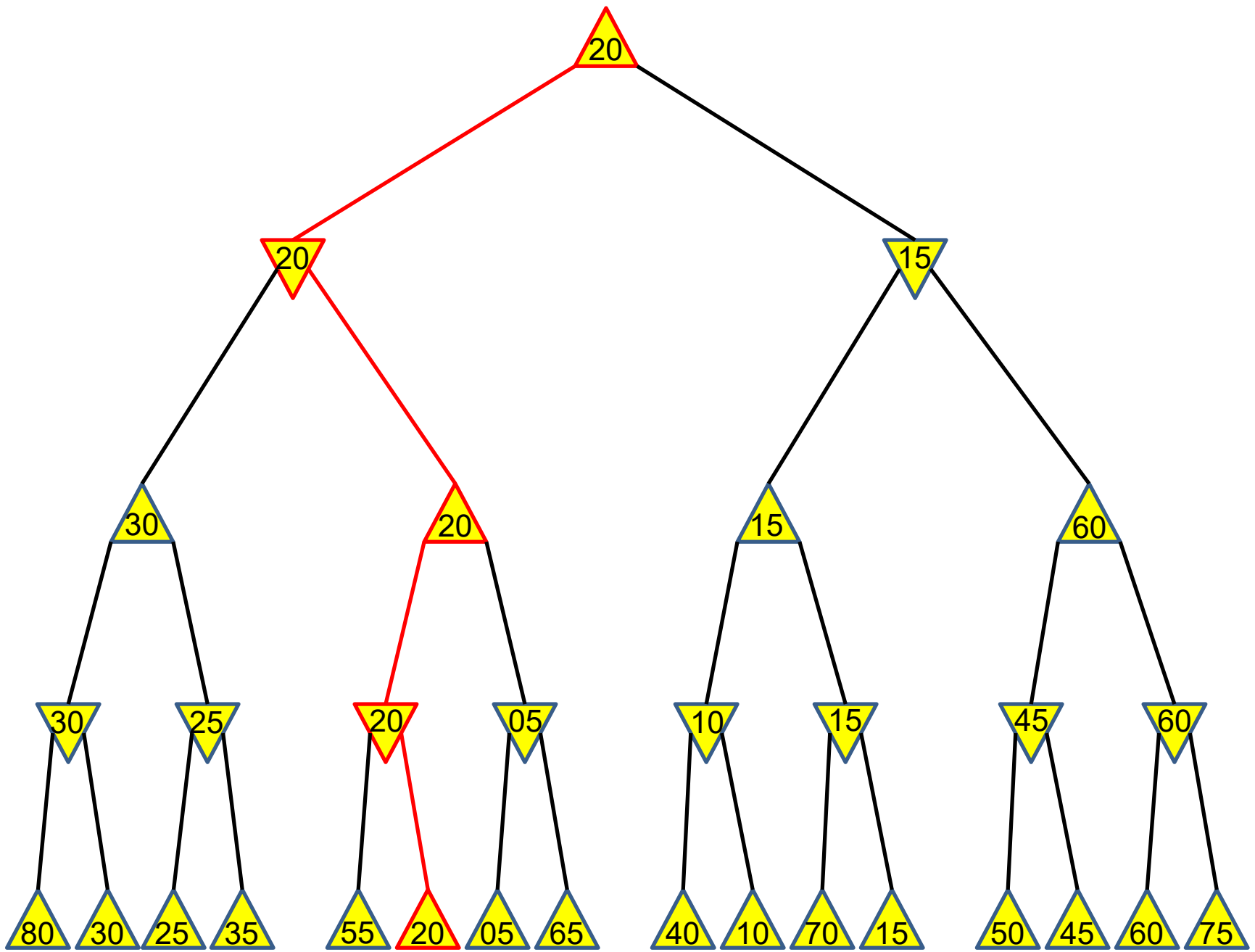












Minimax Strategy

- Why do we take the **min** value every other level of the tree?
- These nodes represent the **opponent's** choice of move.
- The computer assumes that the human will choose that move that is of **least value** to the computer.

Minimax algorithm

Adversarial analogue of DFS

function MINIMAX-DECISION(*state*) *returns an action*

$v \leftarrow \text{MAX-VALUE}(\textit{state})$

return the *action* in SUCCESSORS(*state*) with value *v*

function MAX-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$

return *v*

function MIN-VALUE(*state*) *returns a utility value*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow \infty$

for *a, s* in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$

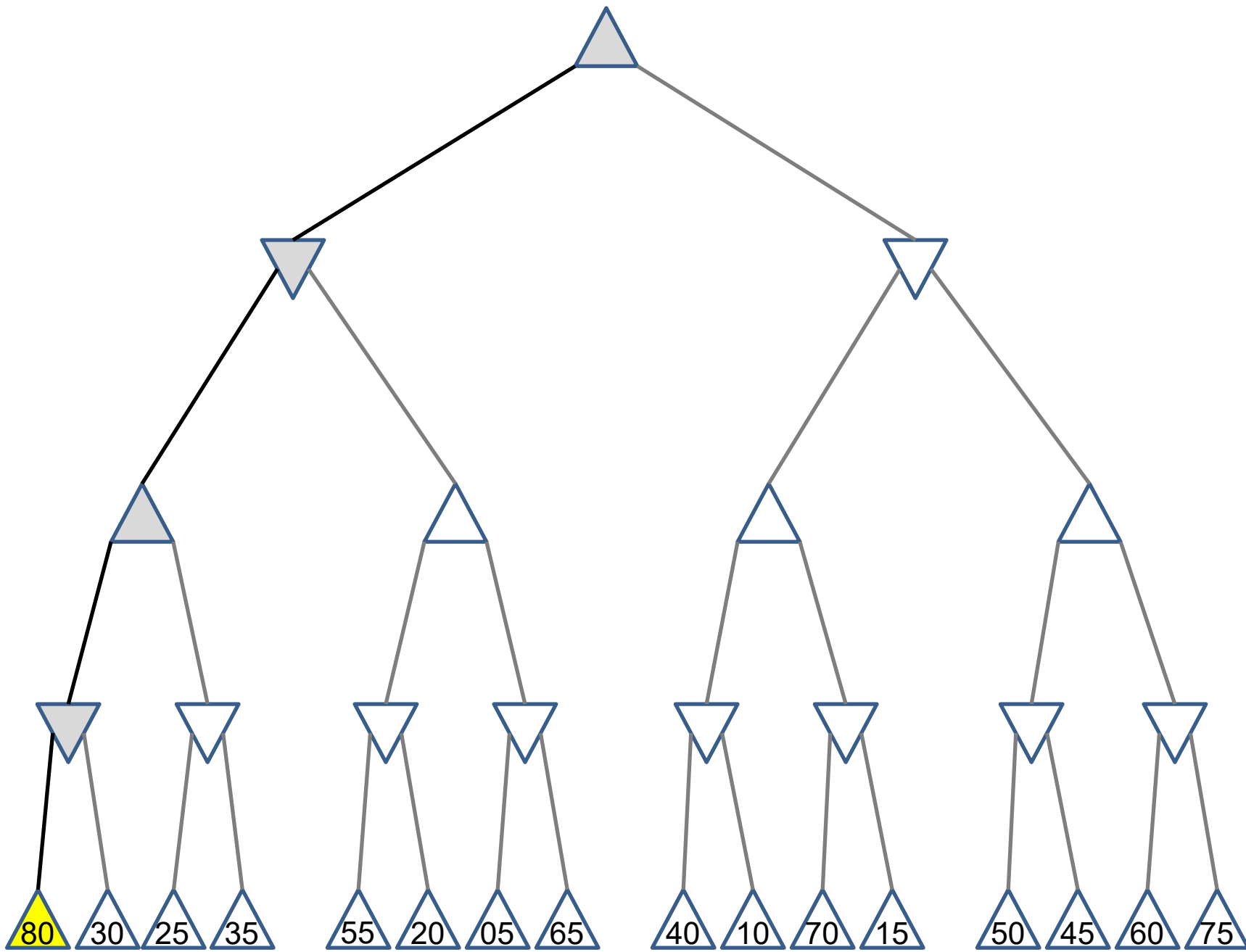
return *v*

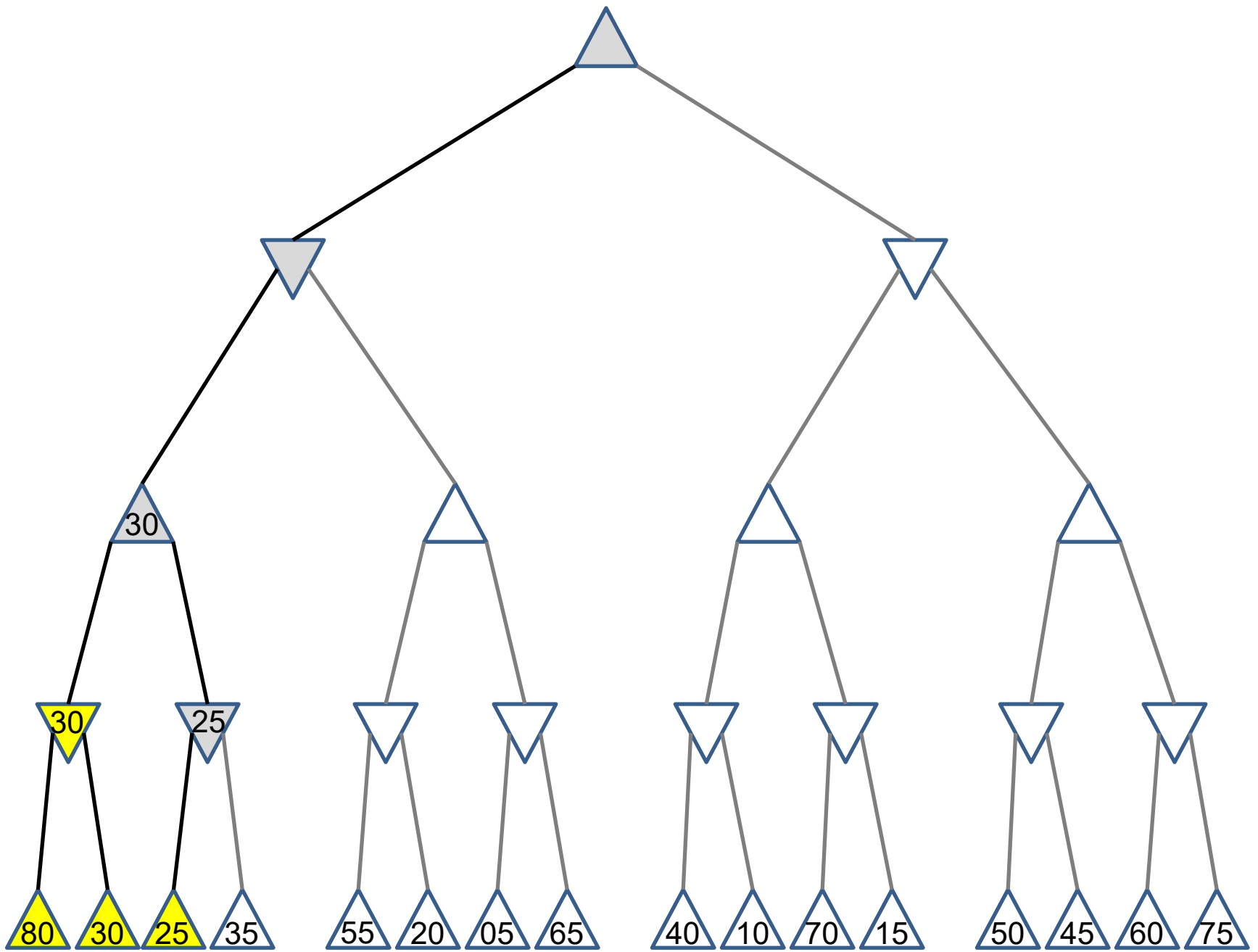
Properties of Minimax

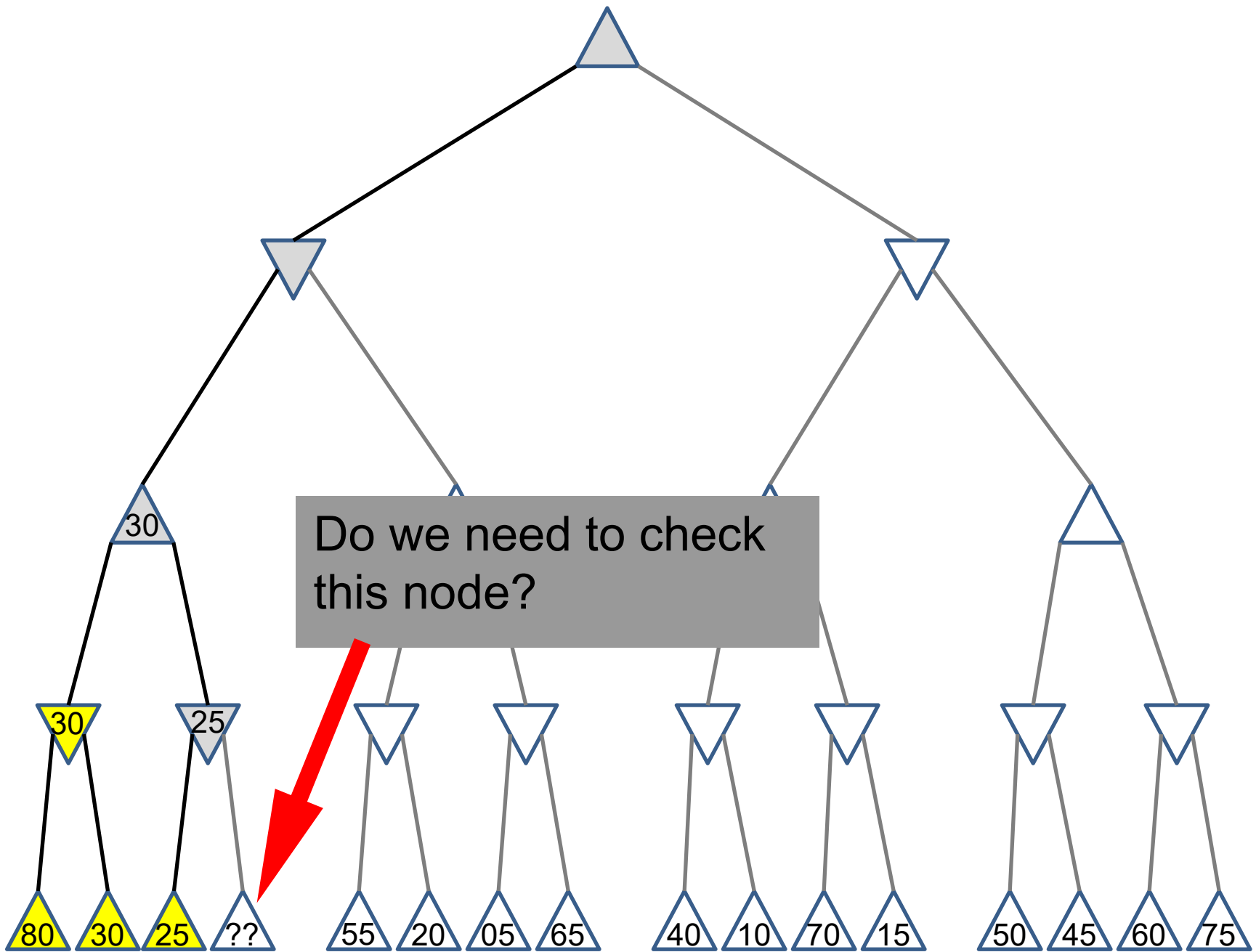
- Complete?
 - Yes (if tree is finite)
- Optimal?
 - Yes (against an optimal opponent)
 - No (does not exploit opponent weakness against suboptimal opponent)
- Time complexity?
 - $O(b^m)$
- Space complexity?
 - $O(bm)$ (depth-first exploration)

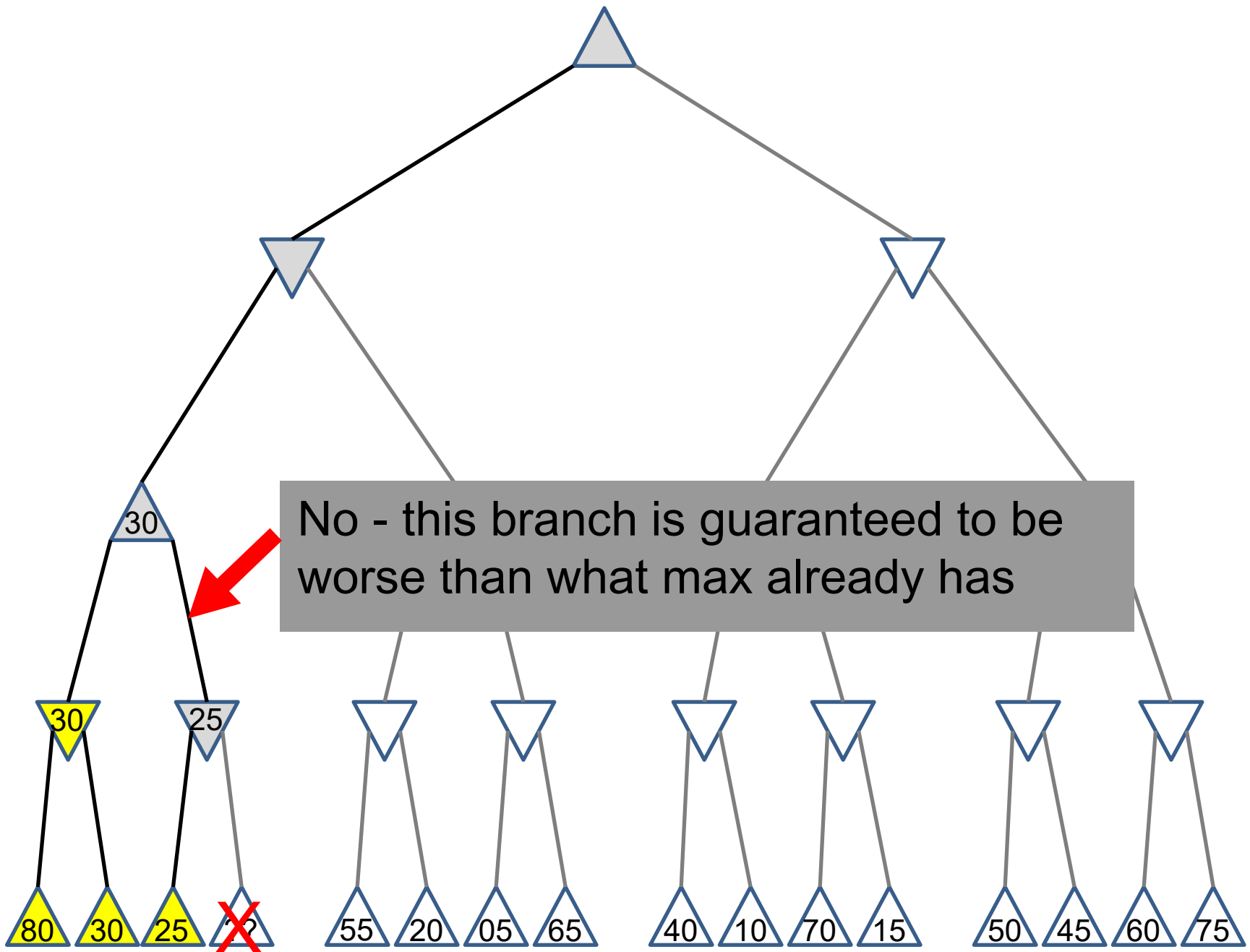
Good Enough?

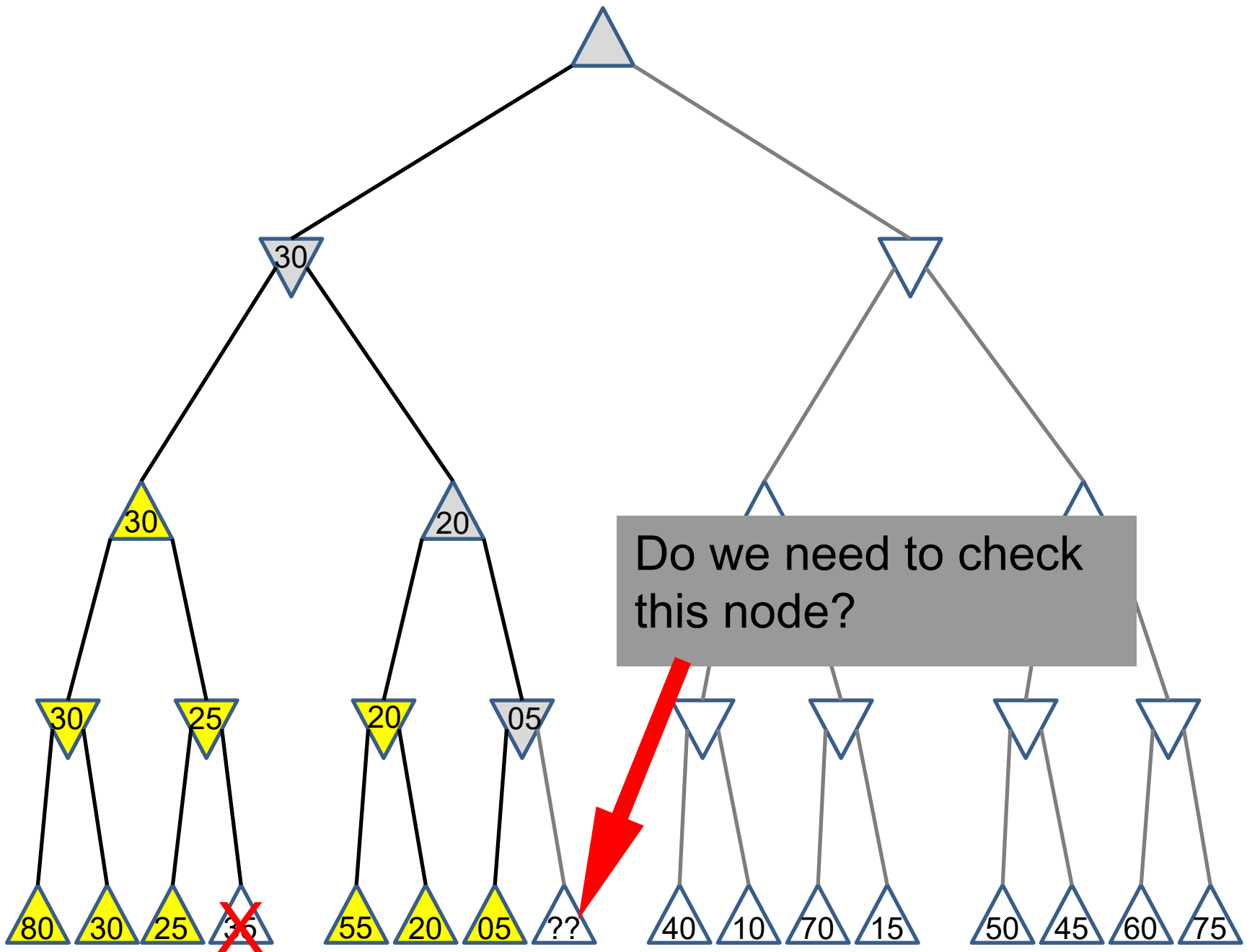
- Chess:
 - branching factor $b \approx 35$
 - game length $m \approx 100$
 - search space $b^m \approx 35^{100} \approx 10^{154}$
- The Universe:
 - number of atoms $\approx 10^{78}$
 - age $\approx 10^{18}$ seconds
 - 10^8 moves/sec $\times 10^{78} \times 10^{18} = 10^{104}$
- Exact solution completely infeasible

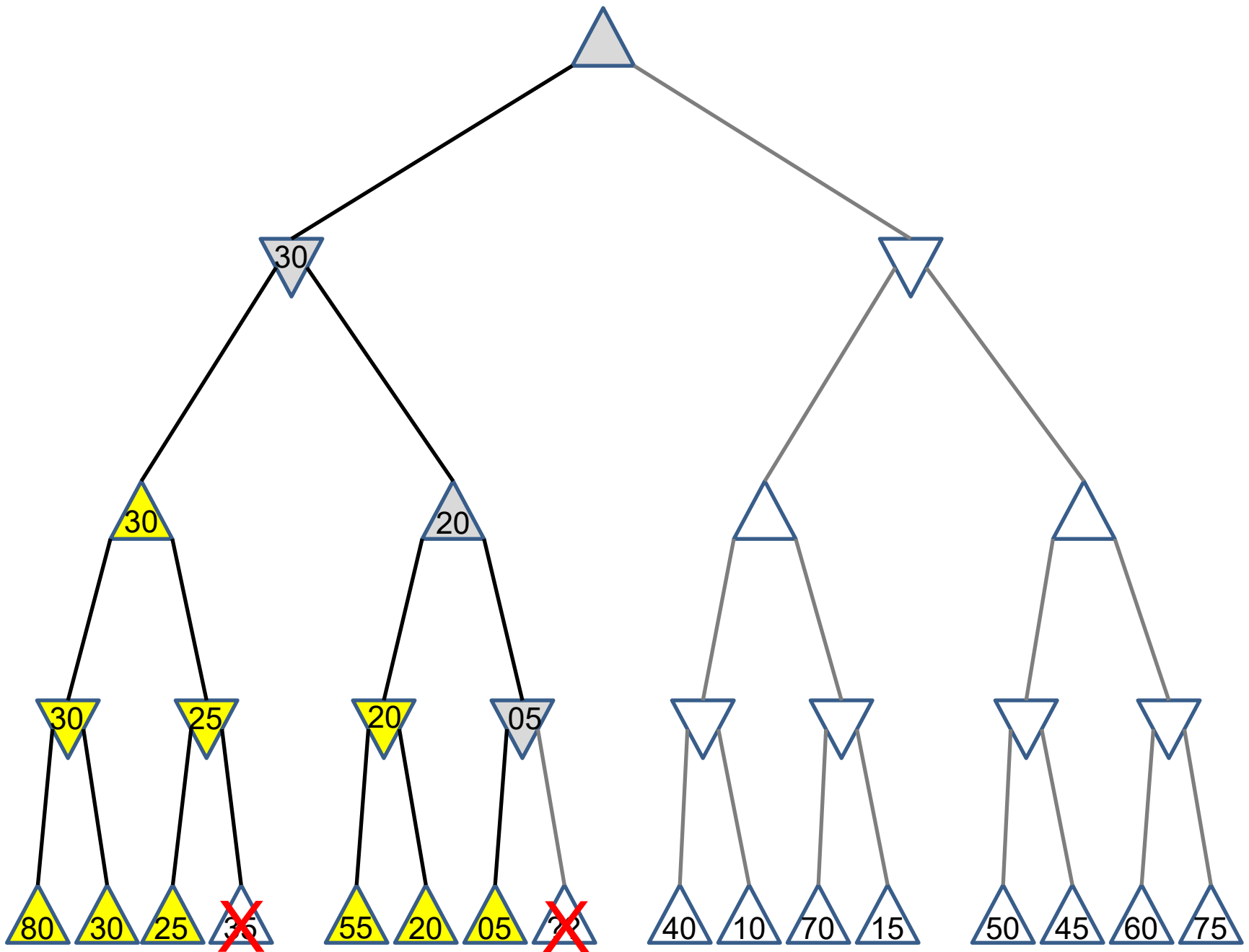












Alpha-Beta

- The alpha-beta procedure can speed up a depth-first minimax search.
- Alpha: a lower bound on the value that a max node may ultimately be assigned

$$v \geq \alpha$$

- Beta: an upper bound on the value that a minimizing node may ultimately be assigned

$$v \leq \beta$$

Alpha-Beta

```
MinVal(state, alpha, beta){  
    if (terminal(state))  
        return utility(state);  
    for (s in children(state)){  
        child = MaxVal(s,alpha,beta);  
        beta = min(beta,child);  
        if (alpha>=beta) return child;  
    }  
    return best child (min); }
```

alpha = the **highest** value for **MAX** along the path

beta = the **lowest** value for **MIN** along the path

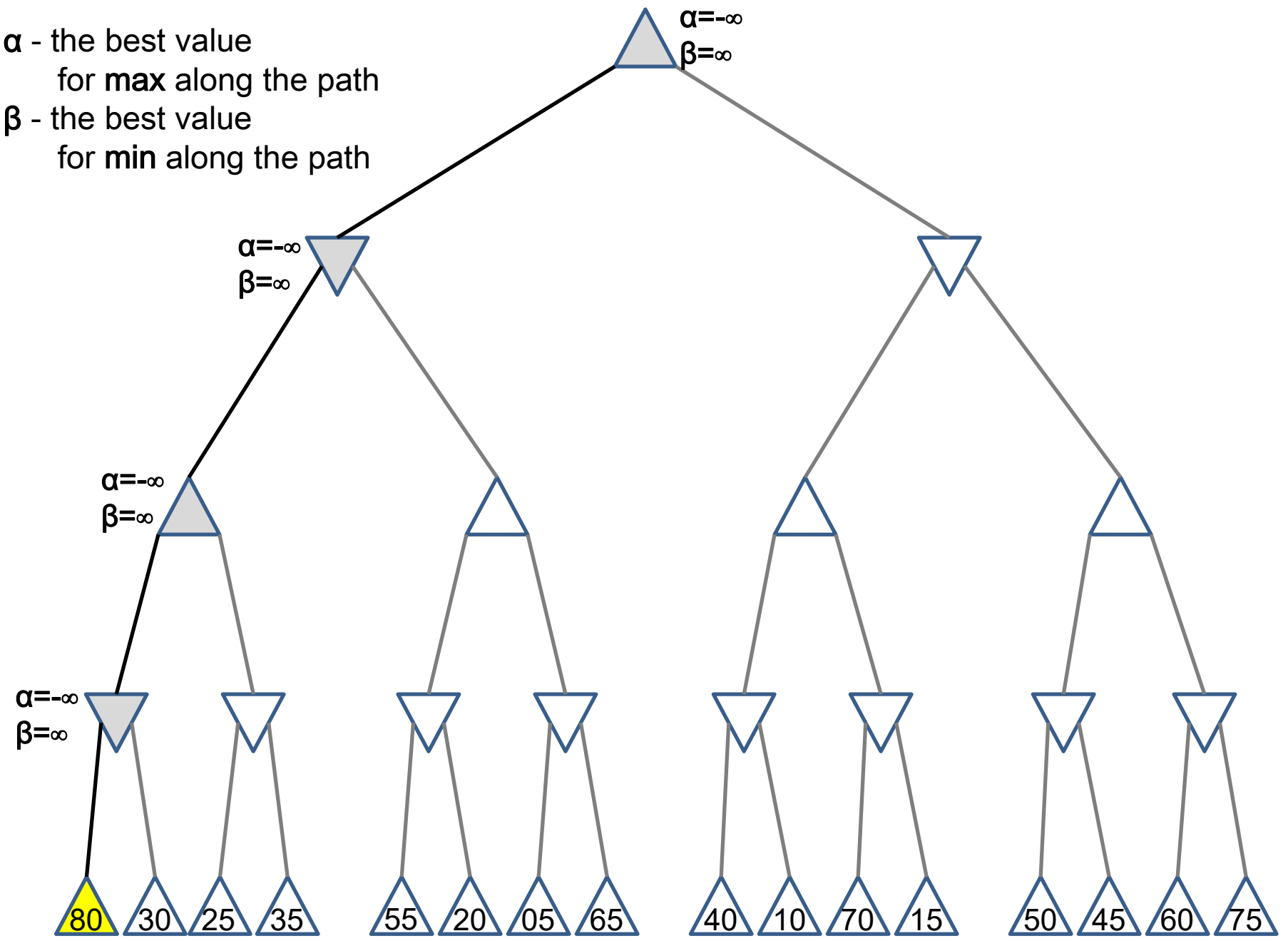
Alpha-Beta

```
MaxVal(state, alpha, beta){  
    if (terminal(state))  
        return utility(state);  
    for (s in children(state)){  
        child = MinVal(s, alpha, beta);  
        alpha = max(alpha, child);  
        if (alpha >= beta) return child;  
    }  
    return best child (max); }
```

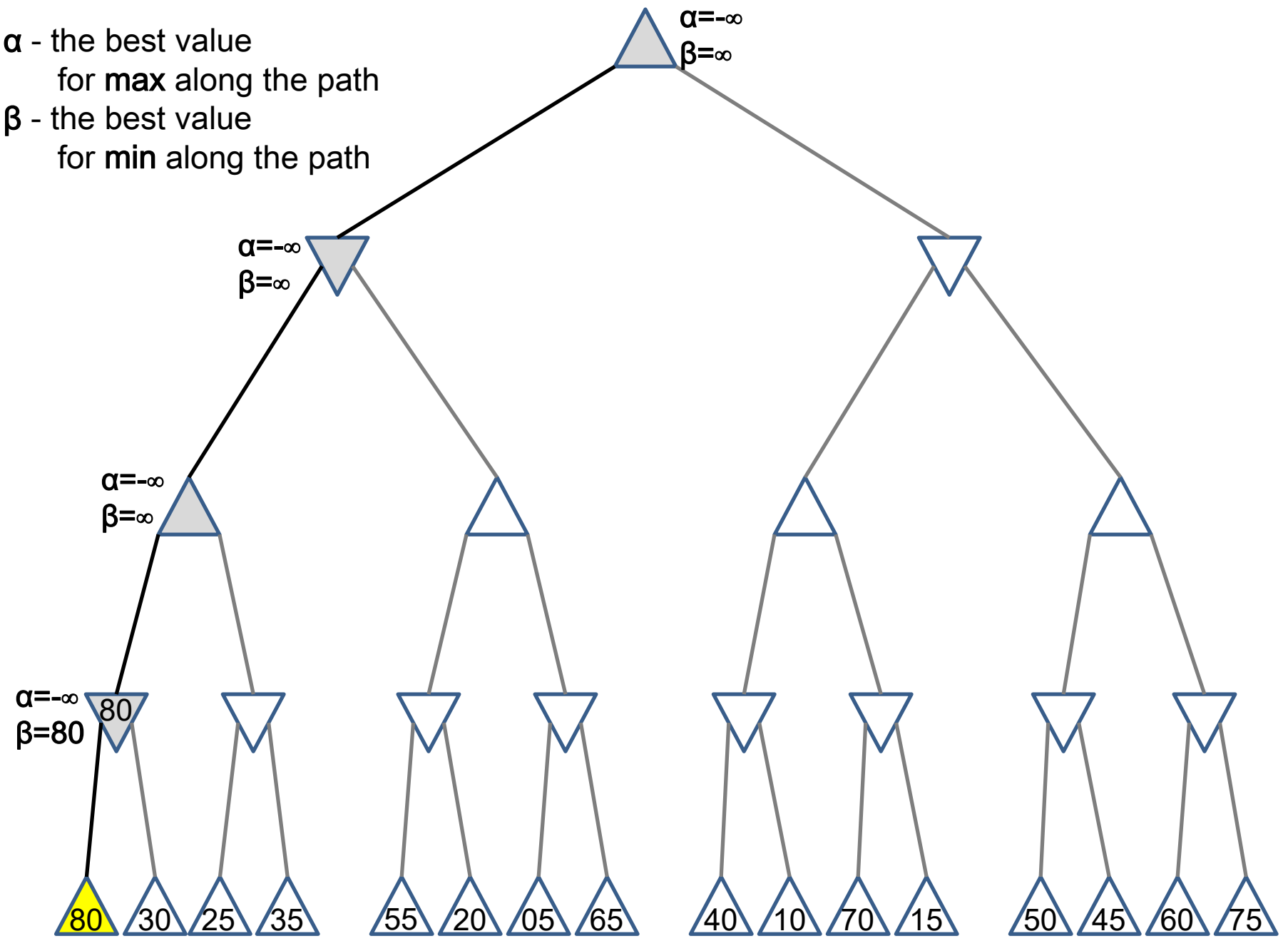
alpha = the highest value for MAX along the path

beta = the lowest value for MIN along the path

α - the best value
for **max** along the path
 β - the best value
for **min** along the path

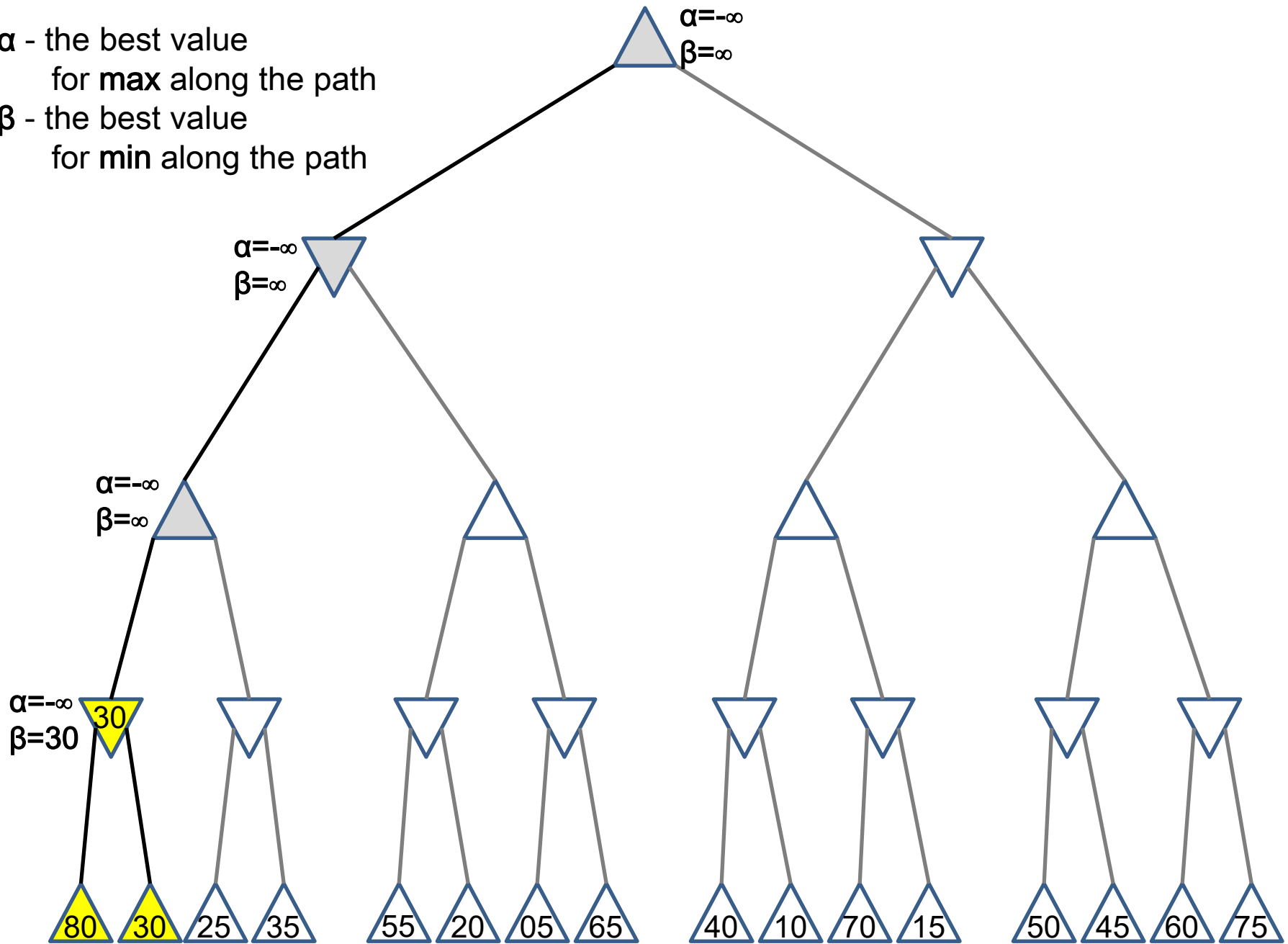


α - the best value
for **max** along the path
 β - the best value
for **min** along the path

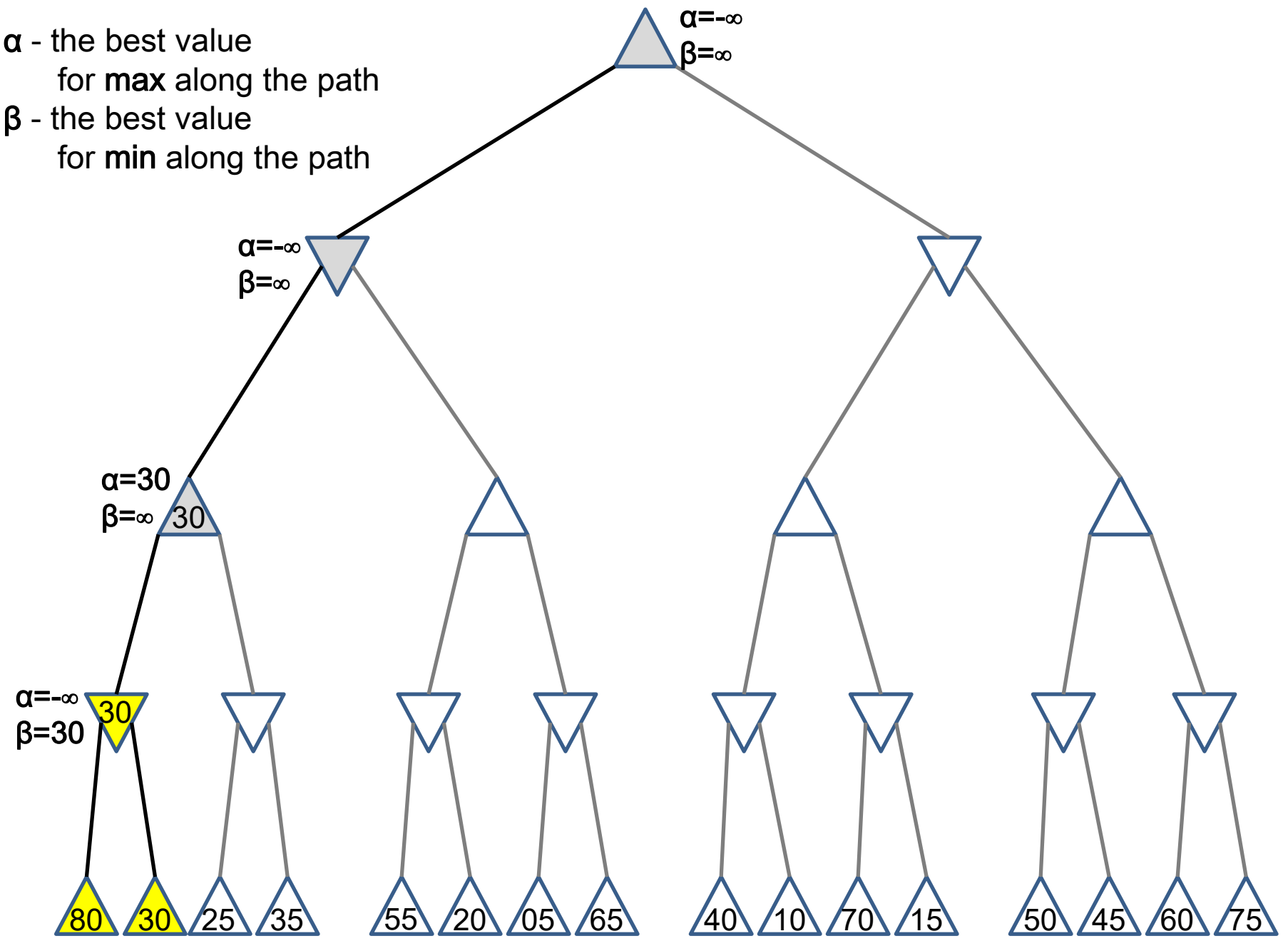


α - the best value
for **max** along the path

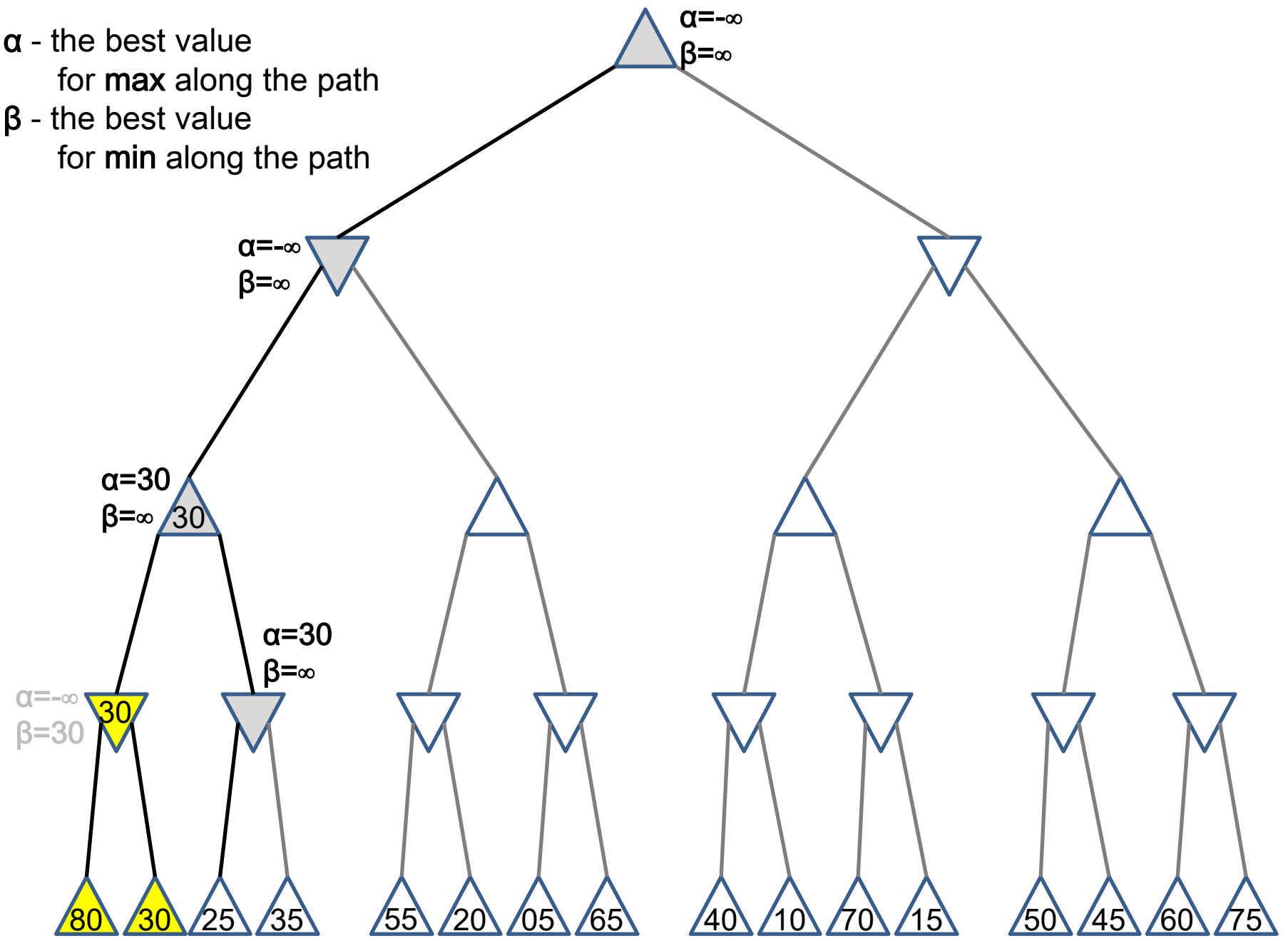
β - the best value
for **min** along the path



α - the best value
for **max** along the path
 β - the best value
for **min** along the path

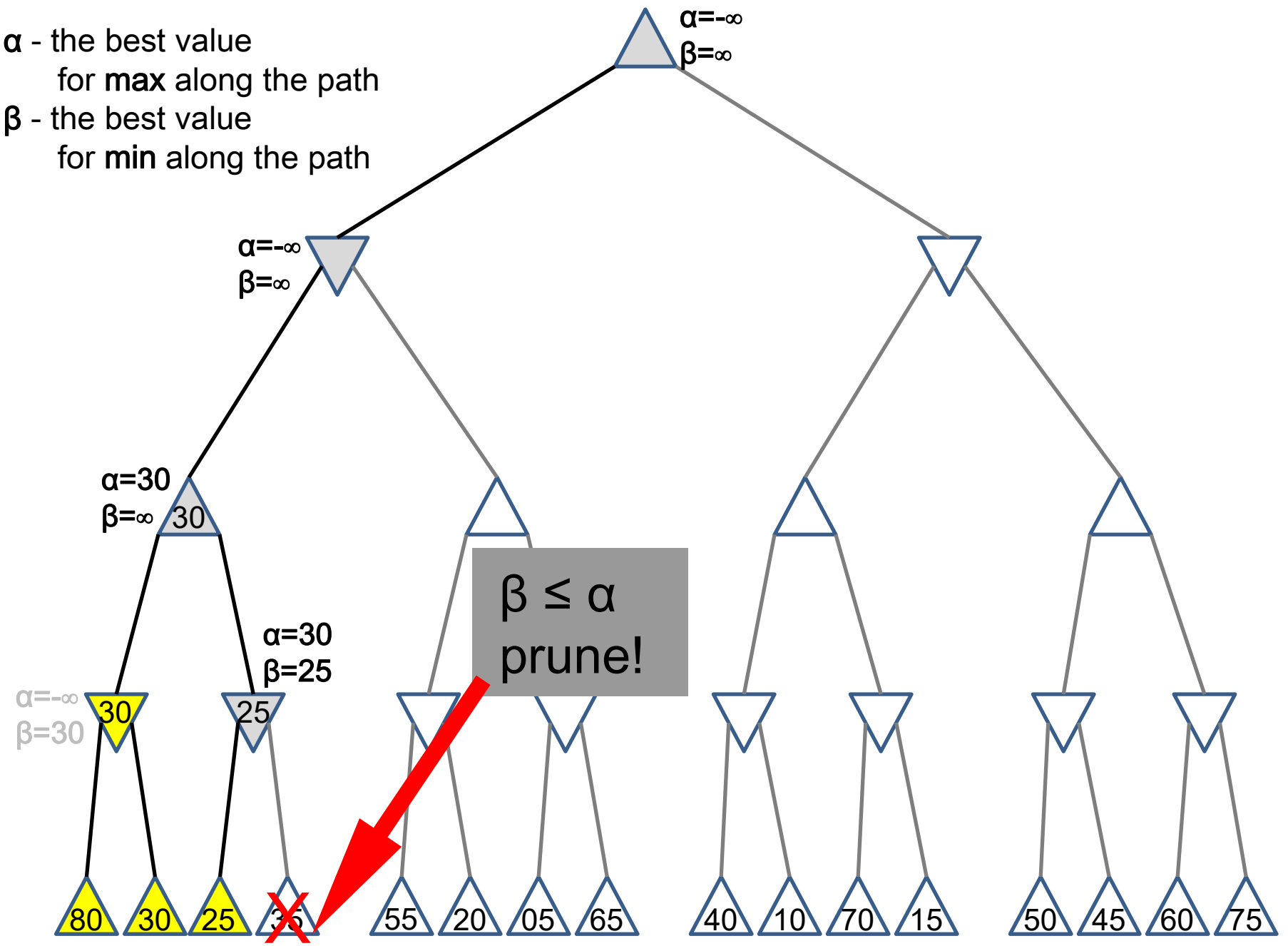


α - the best value
for **max** along the path
 β - the best value
for **min** along the path



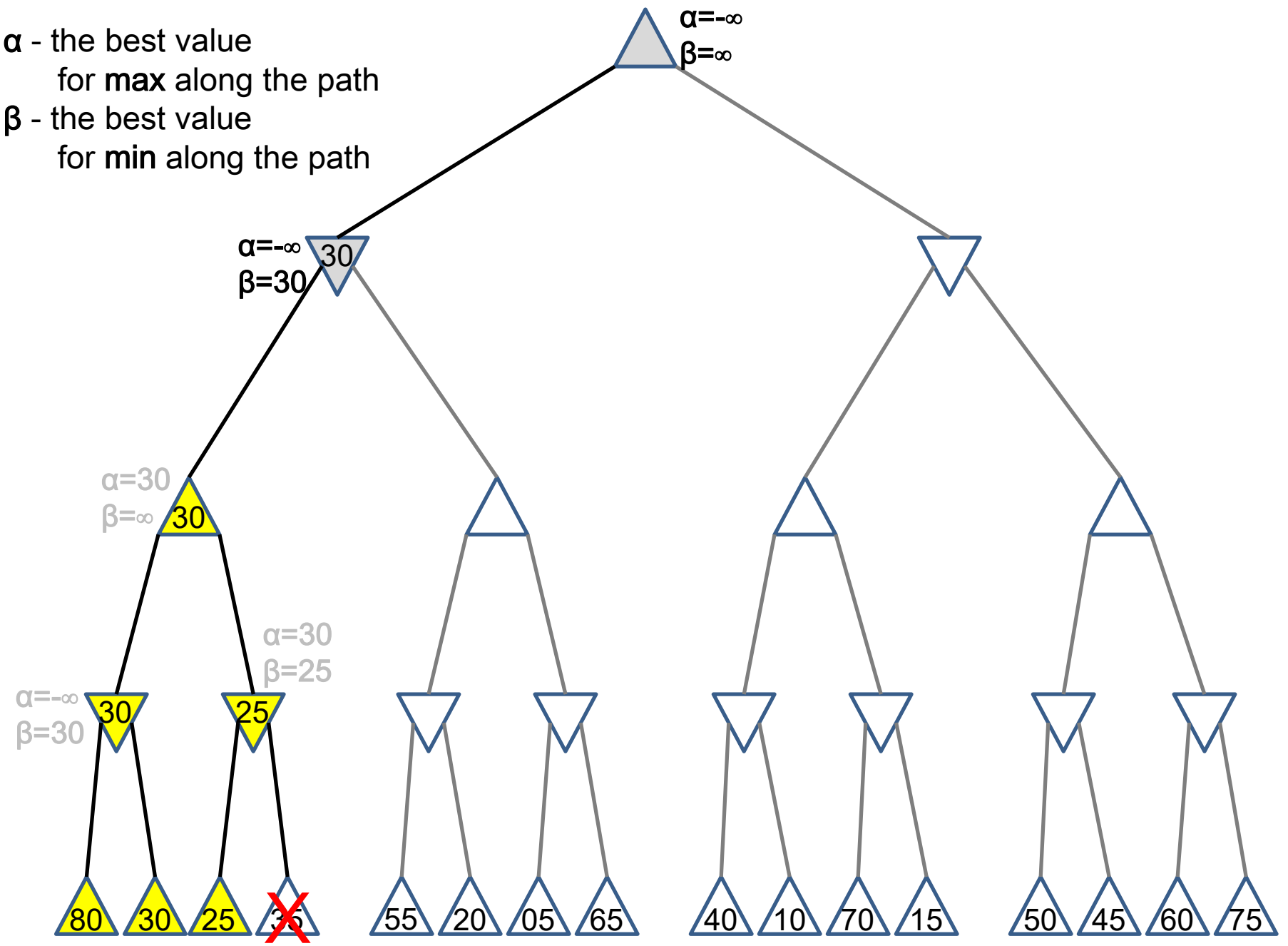
α - the best value
for **max** along the path

β - the best value
for **min** along the path



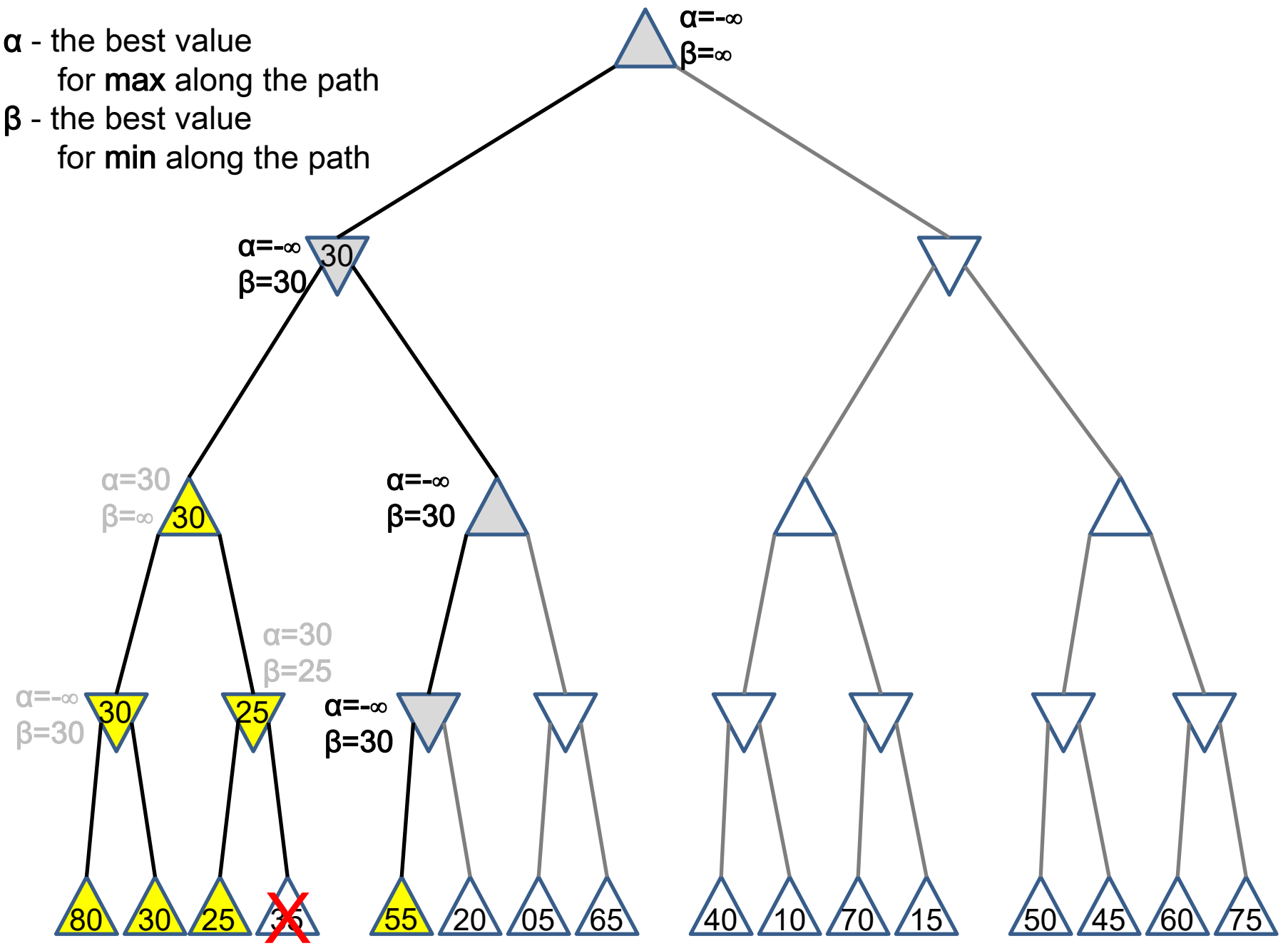
α - the best value
for **max** along the path

β - the best value
for **min** along the path

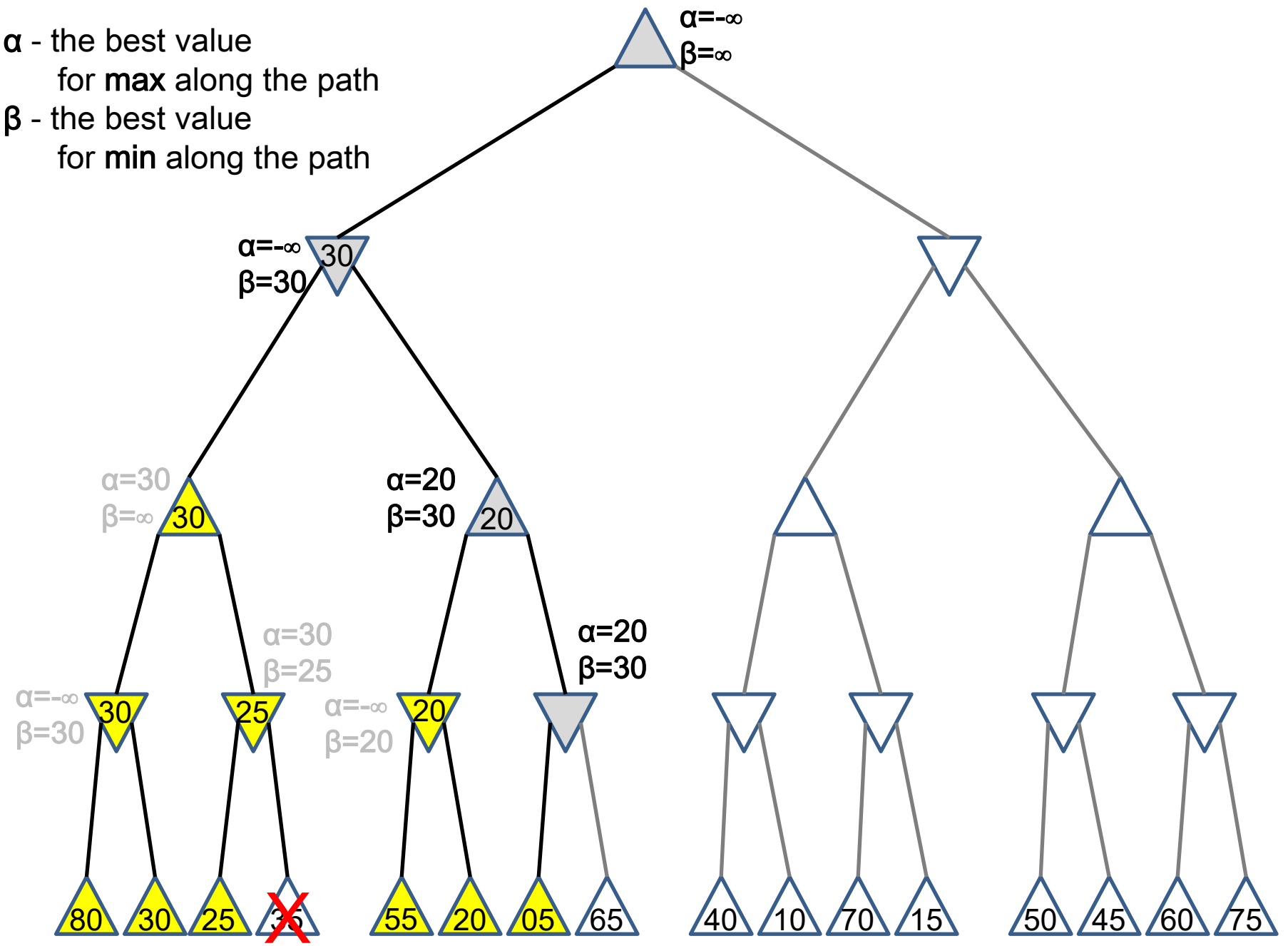


α - the best value
for **max** along the path

β - the best value
for **min** along the path

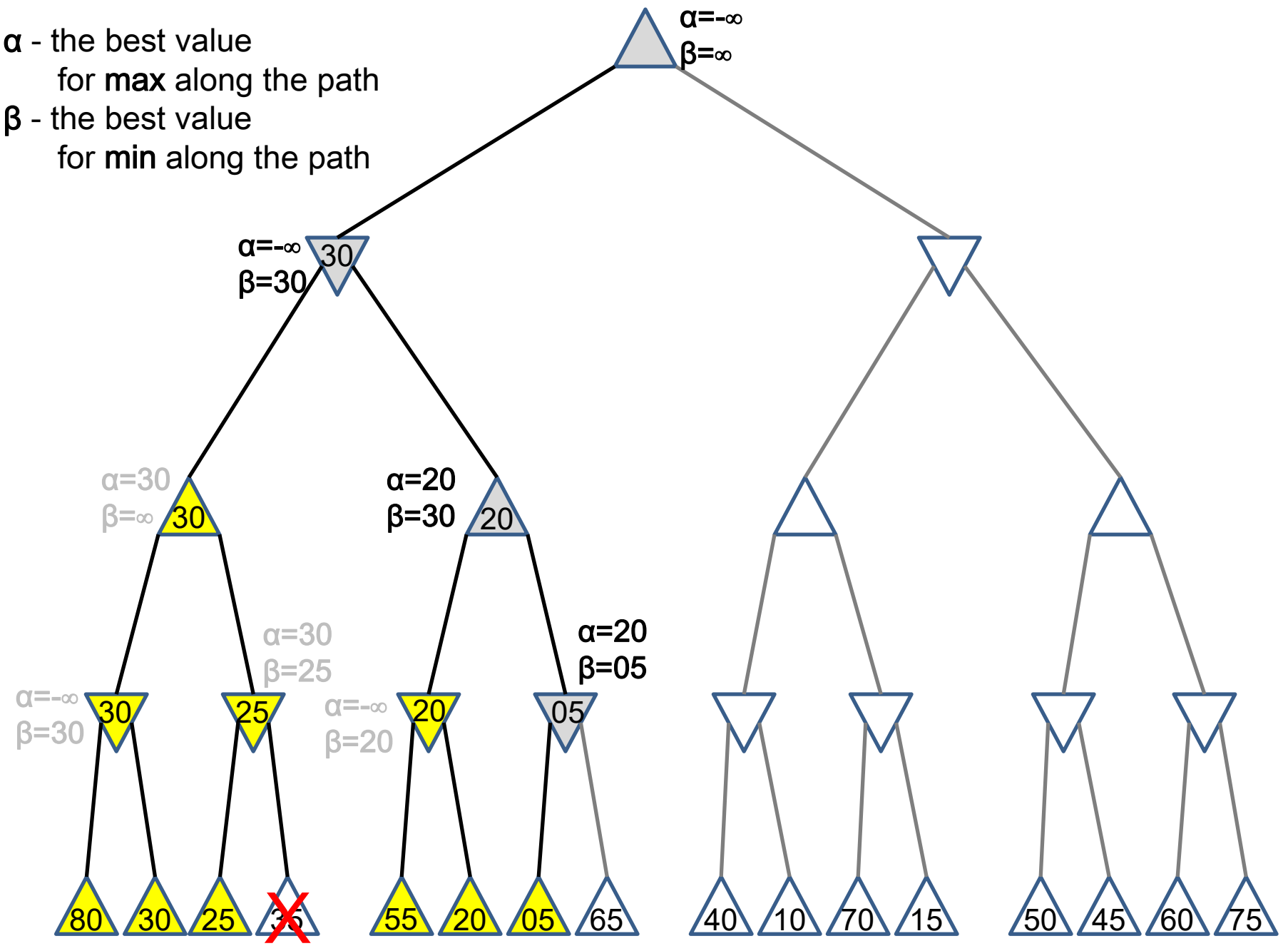


α - the best value
for **max** along the path
 β - the best value
for **min** along the path



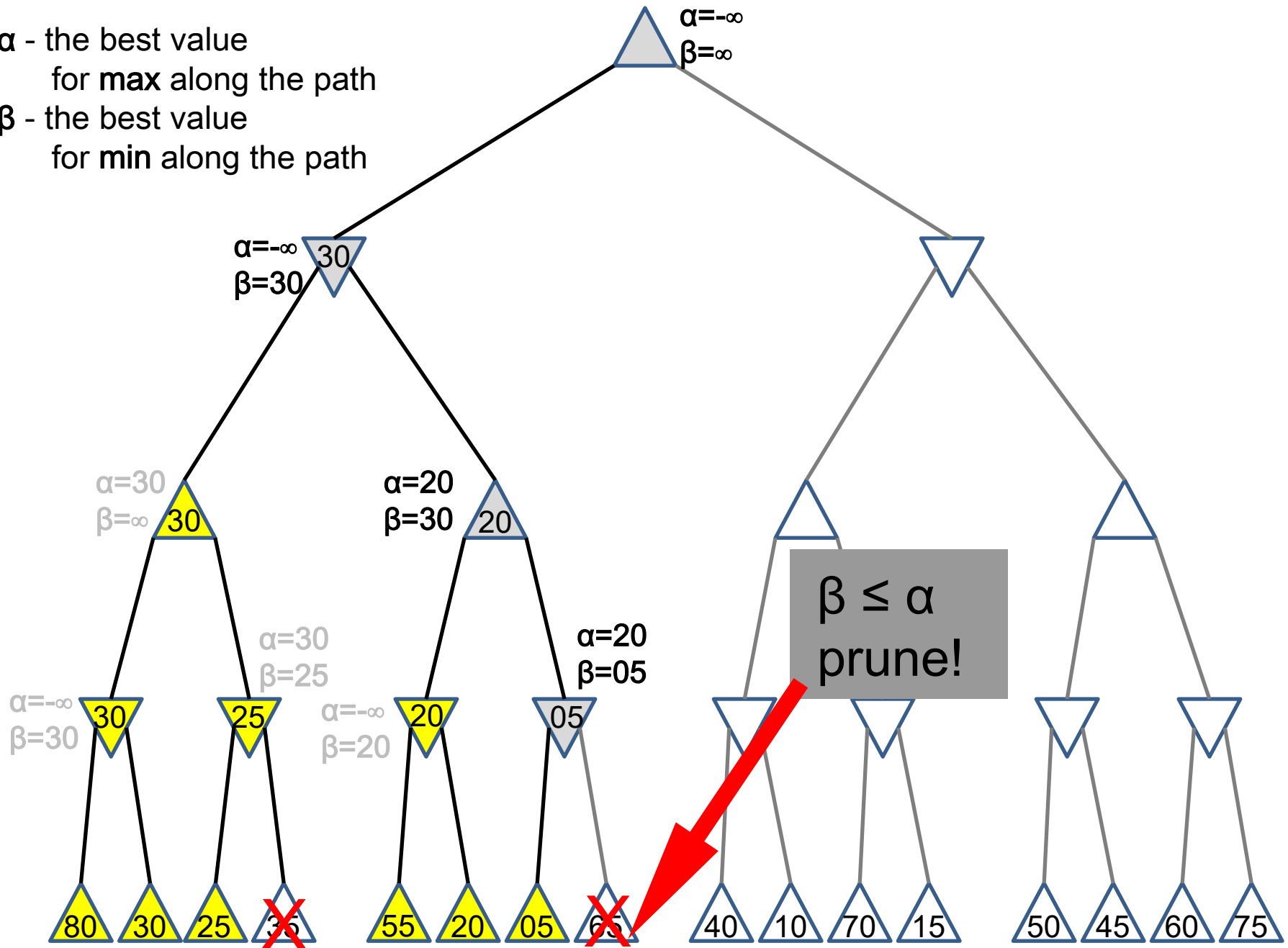
α - the best value
for **max** along the path

β - the best value
for **min** along the path

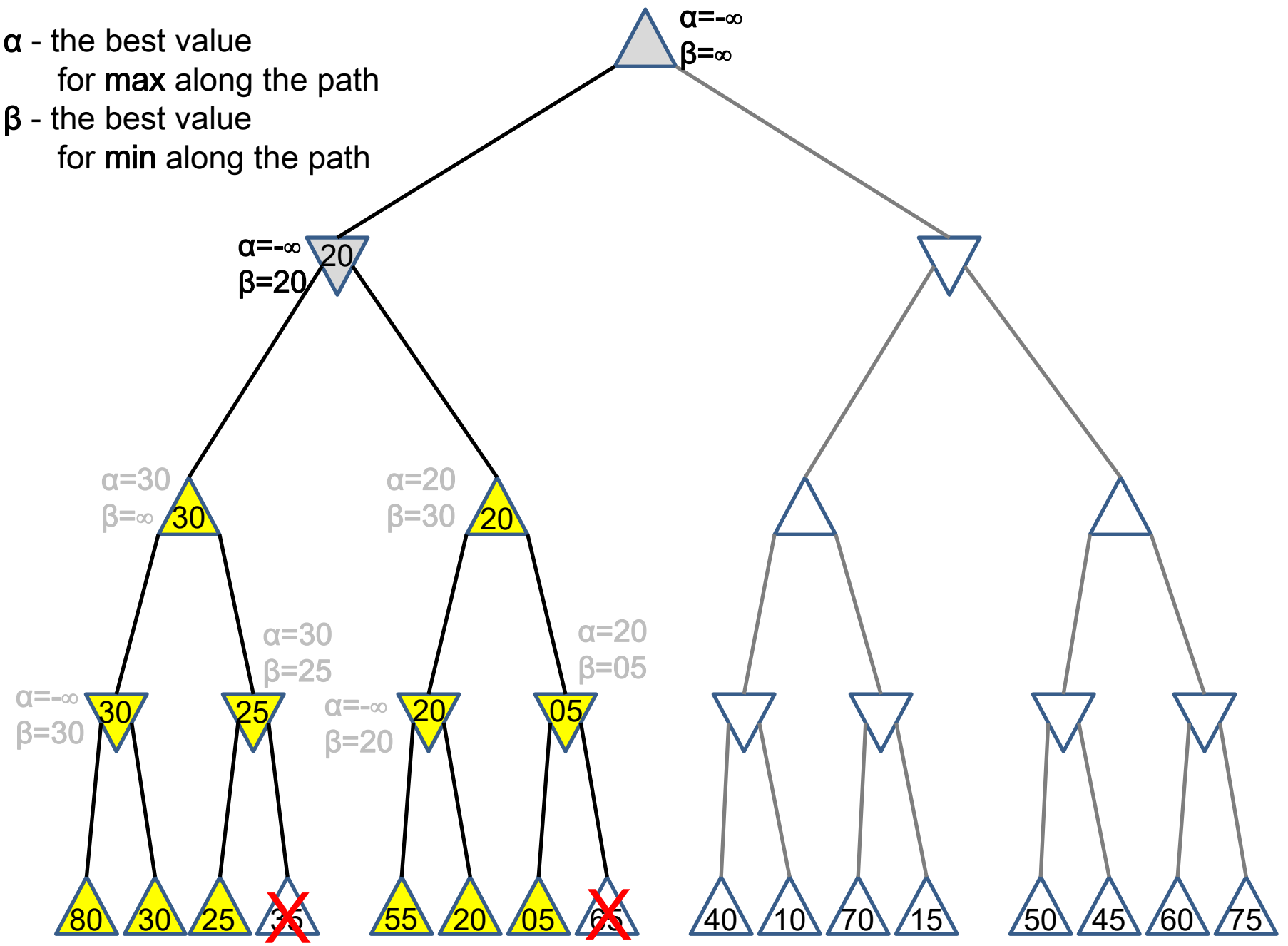


α - the best value
for **max** along the path

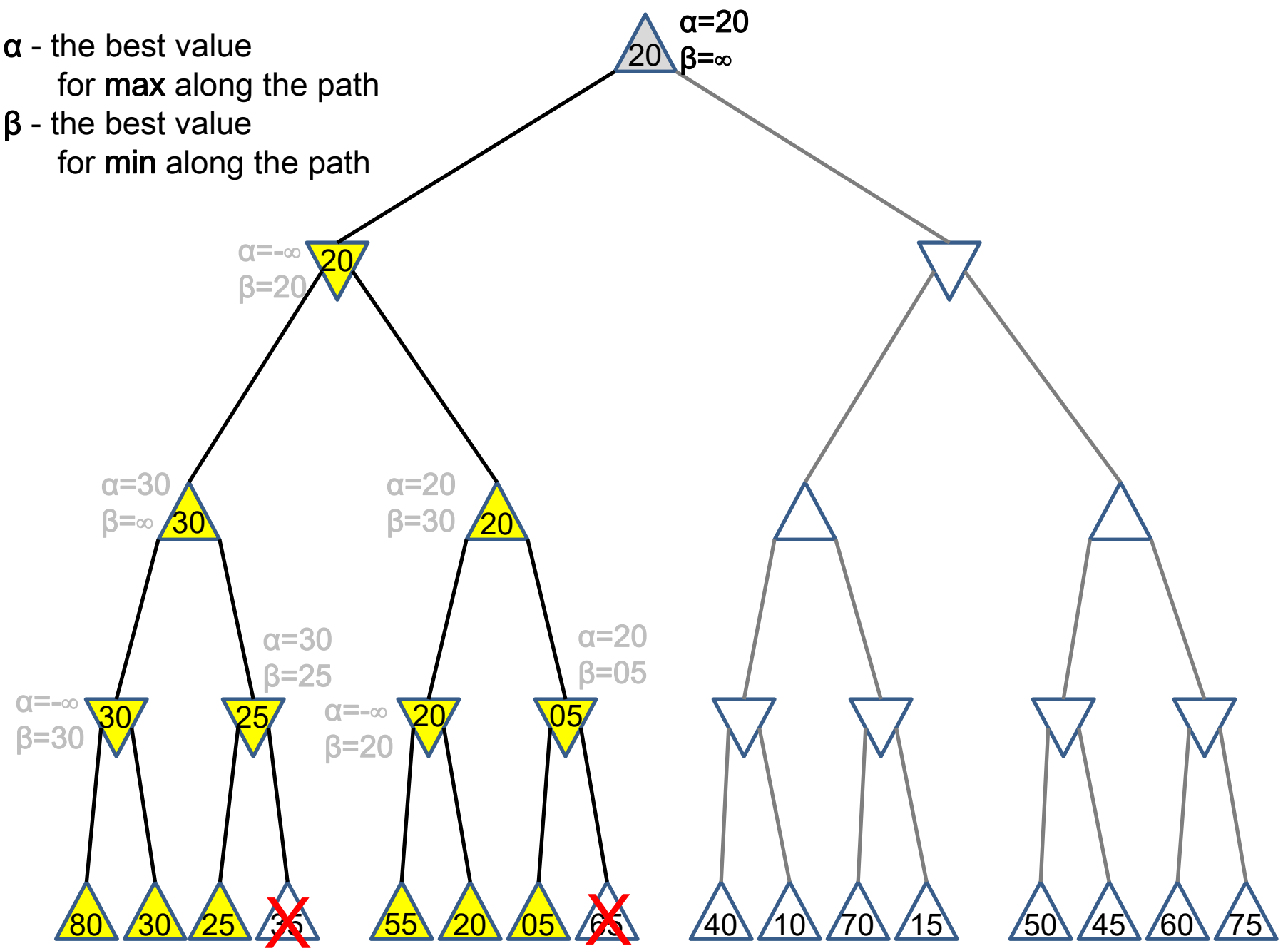
β - the best value
for **min** along the path



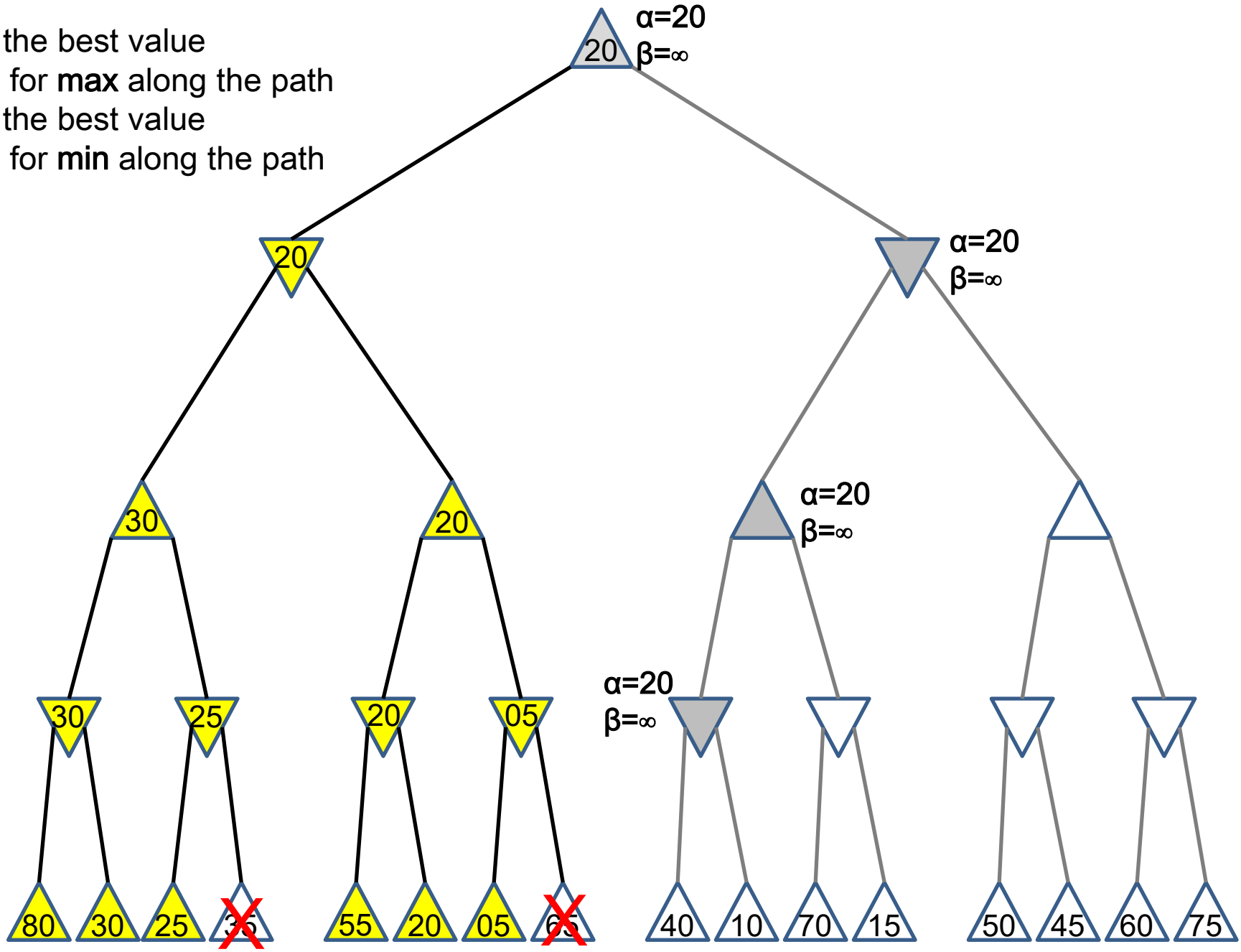
α - the best value
 for **max** along the path
 β - the best value
 for **min** along the path



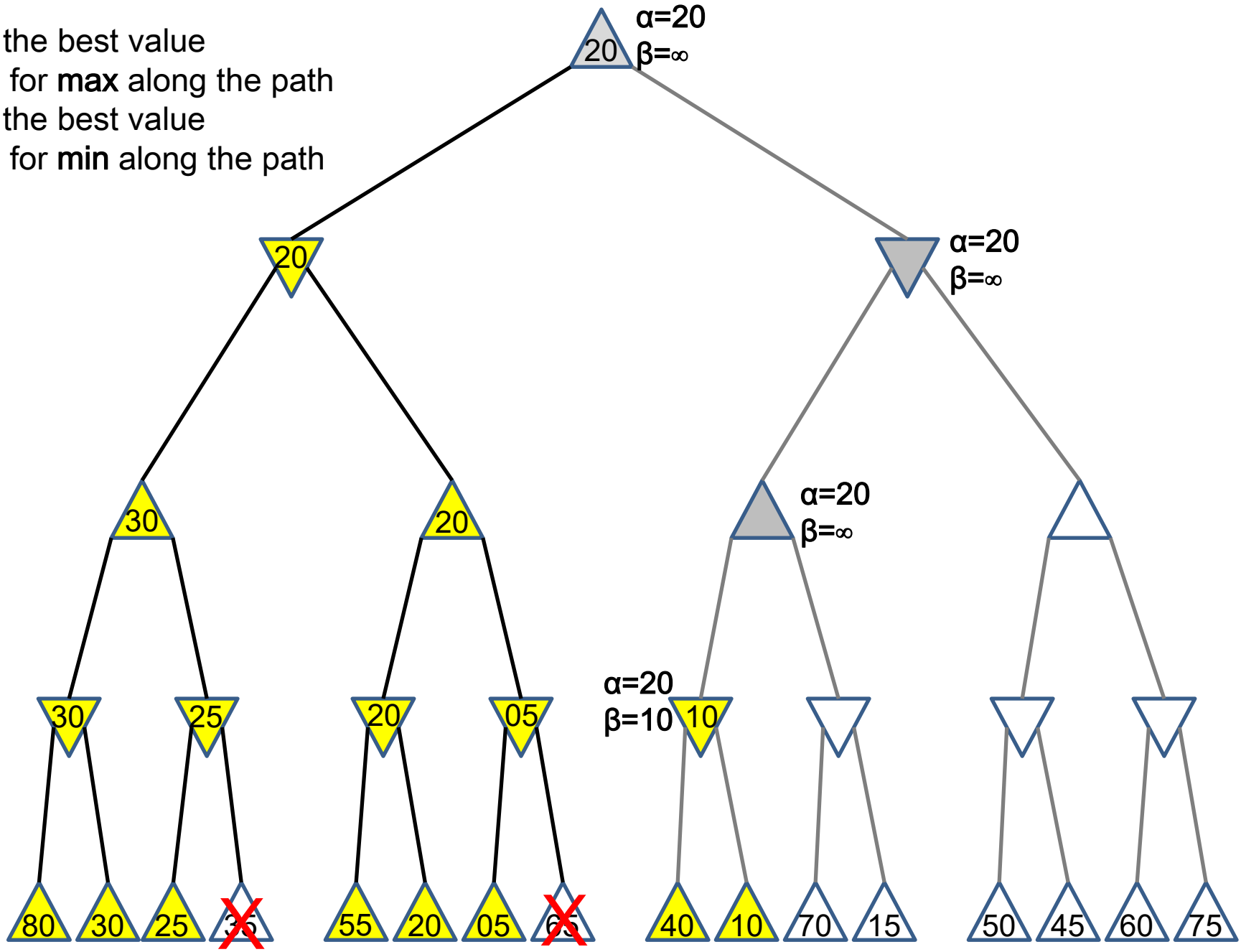
α - the best value
for **max** along the path
 β - the best value
for **min** along the path



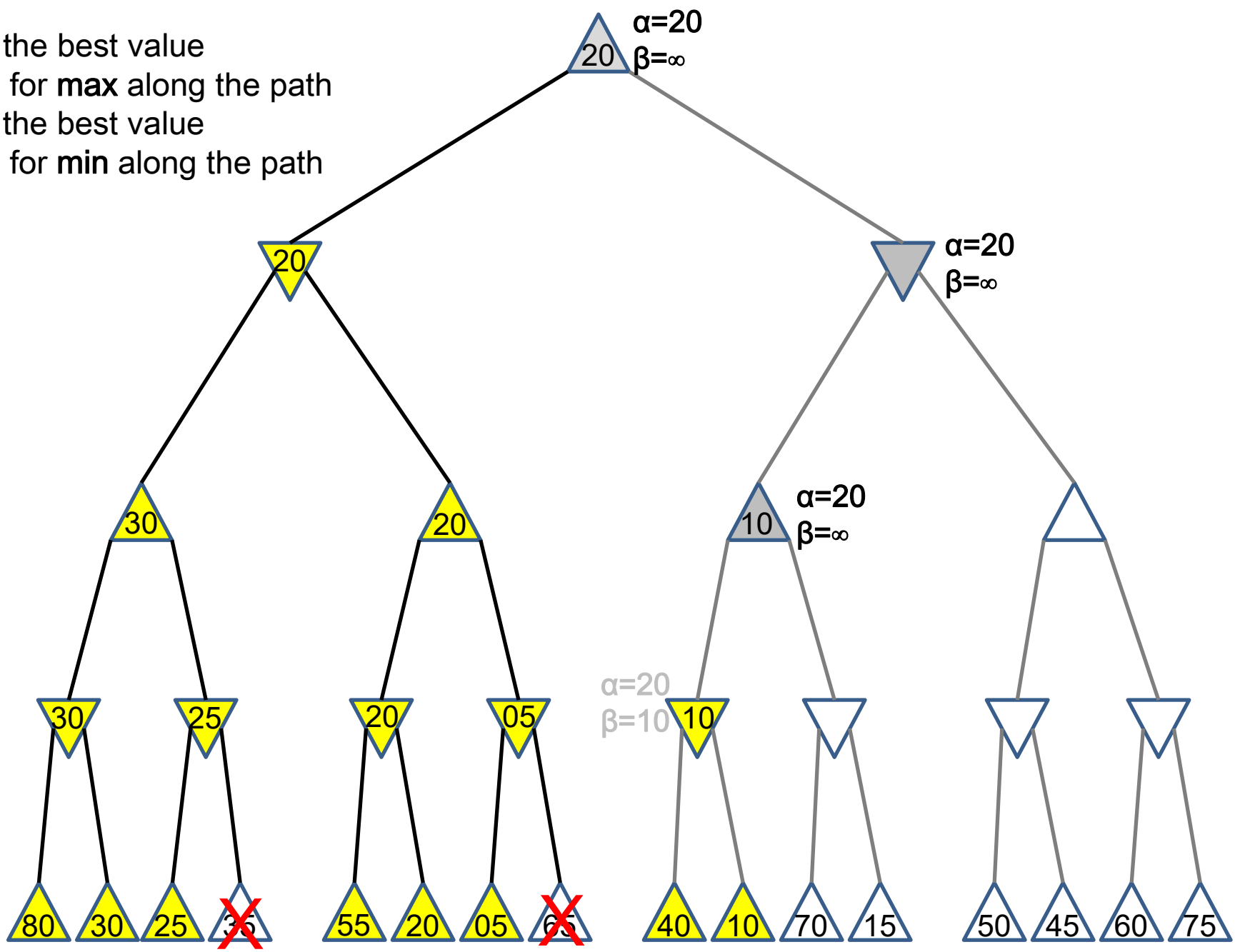
α - the best value
for **max** along the path
 β - the best value
for **min** along the path



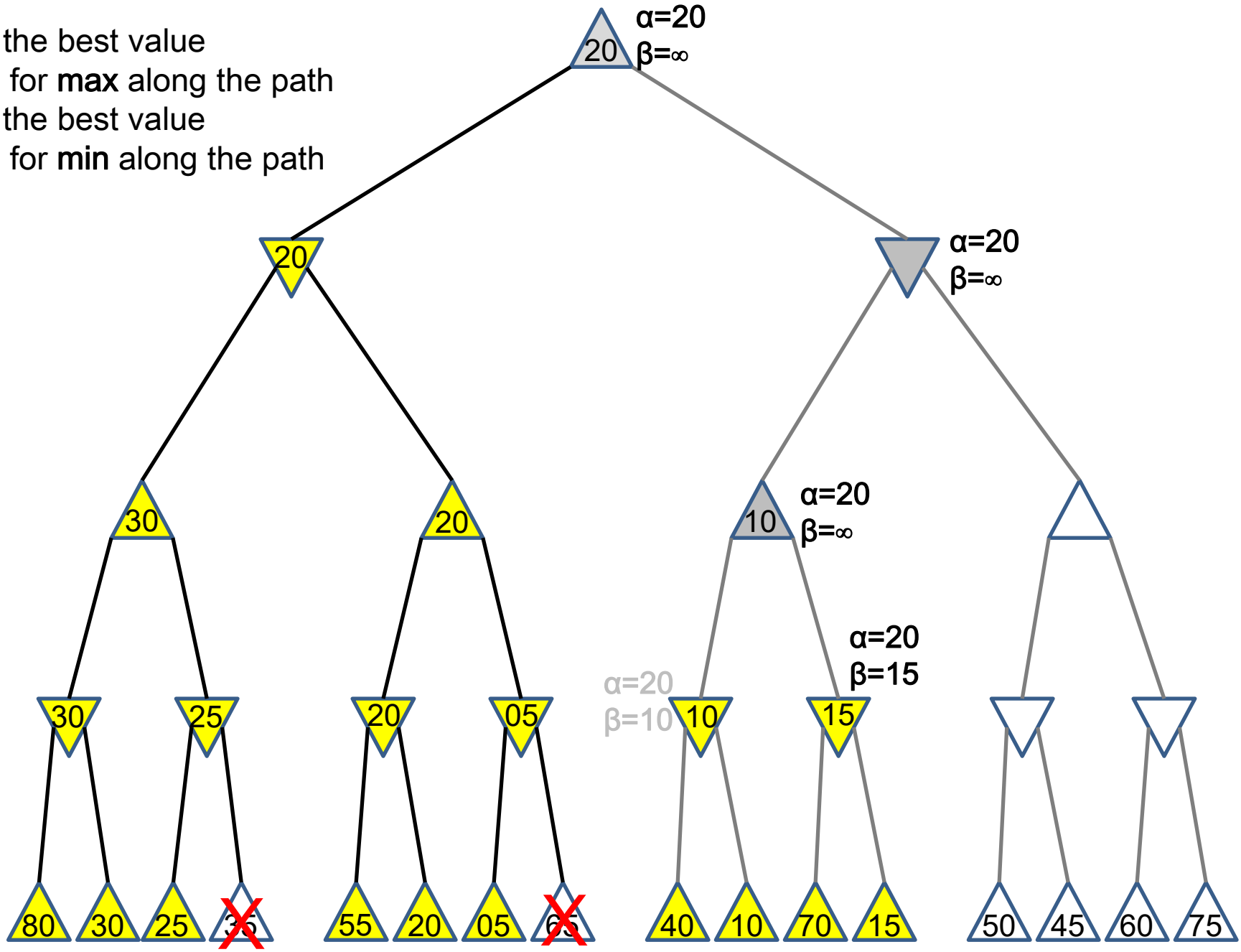
α - the best value
for **max** along the path
 β - the best value
for **min** along the path



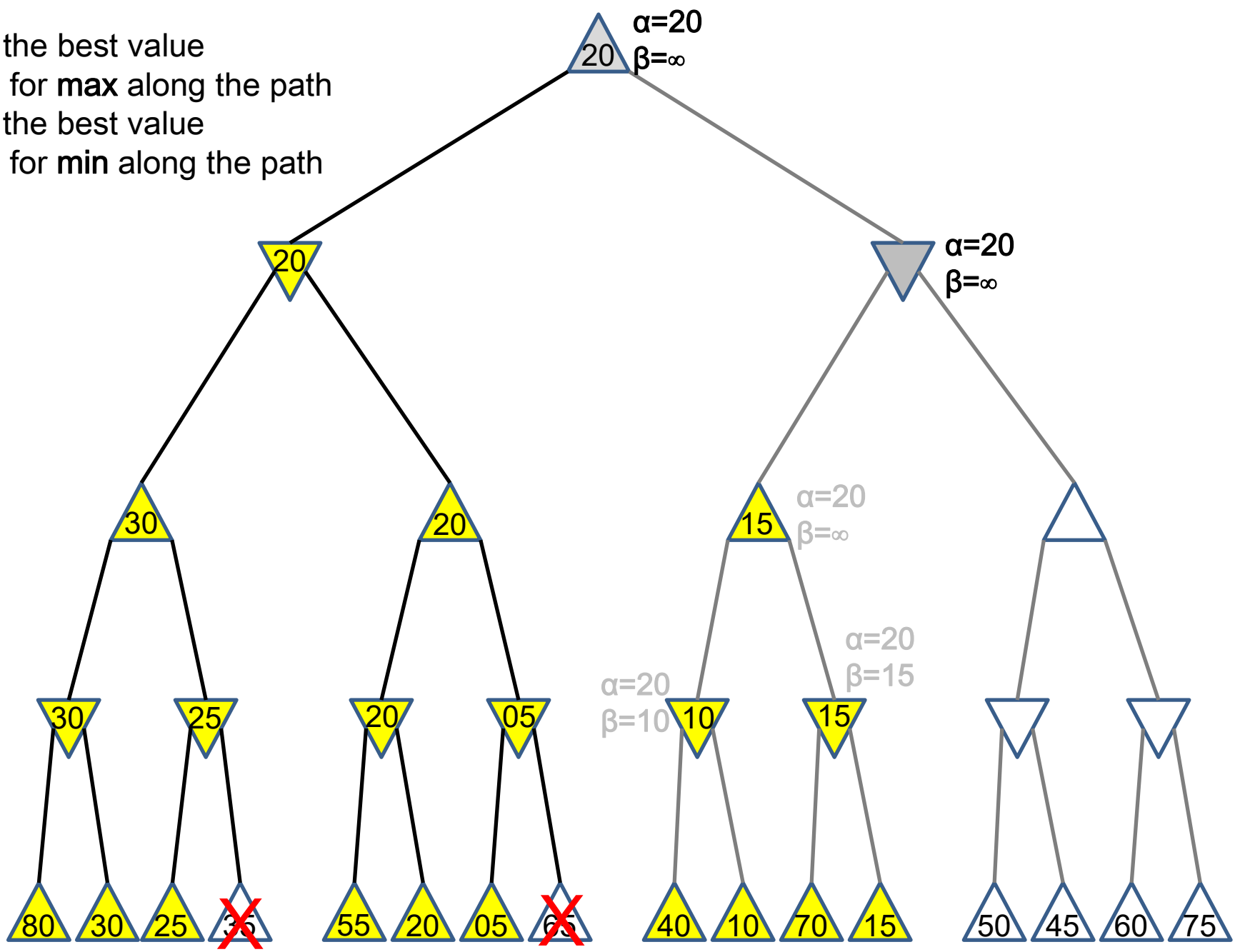
α - the best value
for **max** along the path
 β - the best value
for **min** along the path



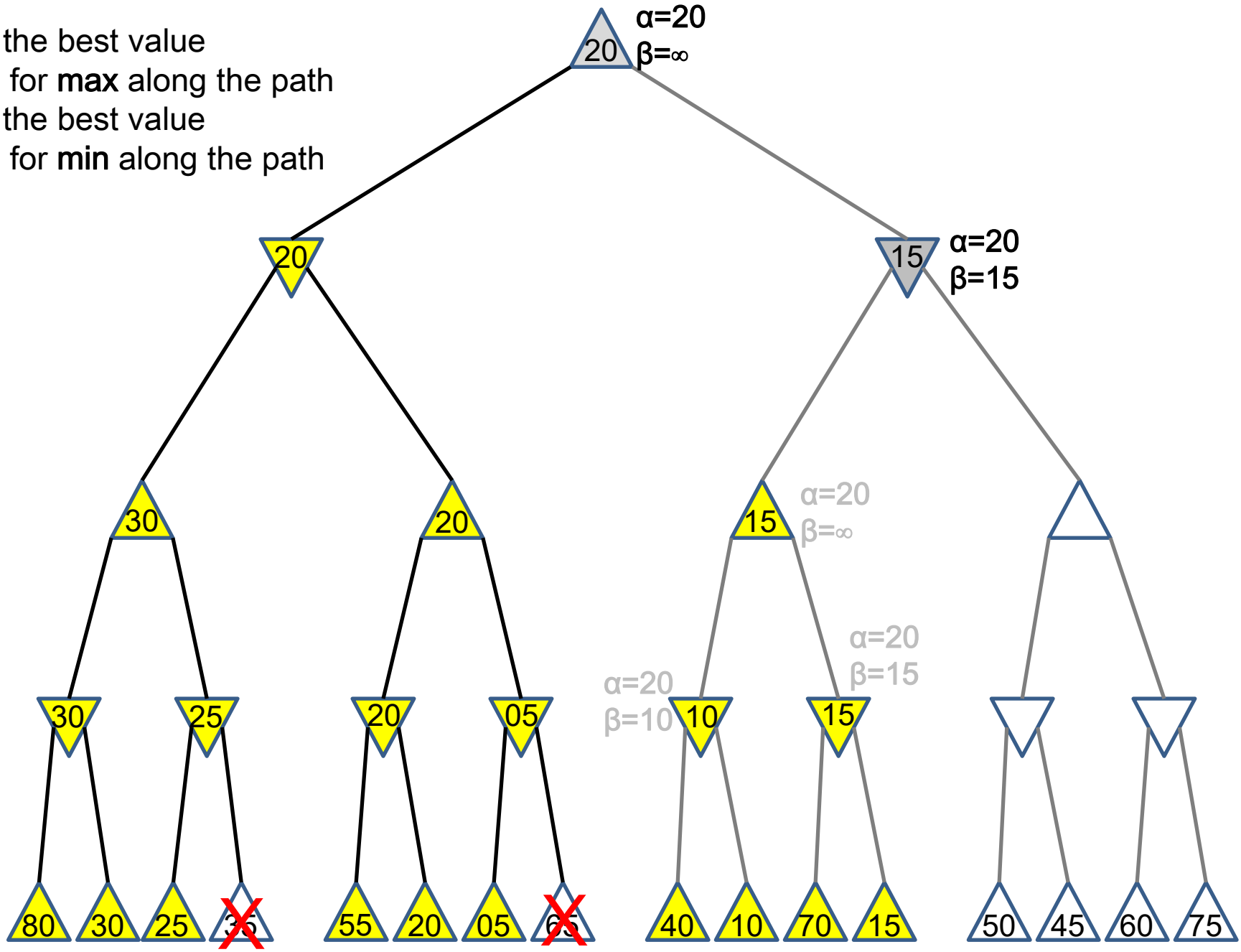
α - the best value
for **max** along the path
 β - the best value
for **min** along the path



α - the best value
for **max** along the path
 β - the best value
for **min** along the path



α - the best value
for **max** along the path
 β - the best value
for **min** along the path



α - the best value
for **max** along the path
 β - the best value
for **min** along the path

